Workshop Kubernetes + Helm + K9s

practice on Harbor Install(s)
(vs Docker-Compose)

arnaud.nauwynck@gmail.com

Outline

- Discover Harbor
- Install…

1/ using « install.sh » => Docker Compose

OR Alternatives, using Kubernetes + Helm

2/ using Linux|WSL (Shell Cmdline) -> Kind -> Helm

OR Alternative

3/ using Podman (GUI) -> Kind -> Helm

- Discover K8s internals (deployment,service,pod,ingress,..)

+ use K9s, and port-forward / exec shell

Discover Harbor

ChatGPT

Free offer ×

Share

explain super shortly Harbor Docker registry features

Harbor is an enterprise container registry built on top of the Docker Registry that adds **security, governance, and multi-registry management**.

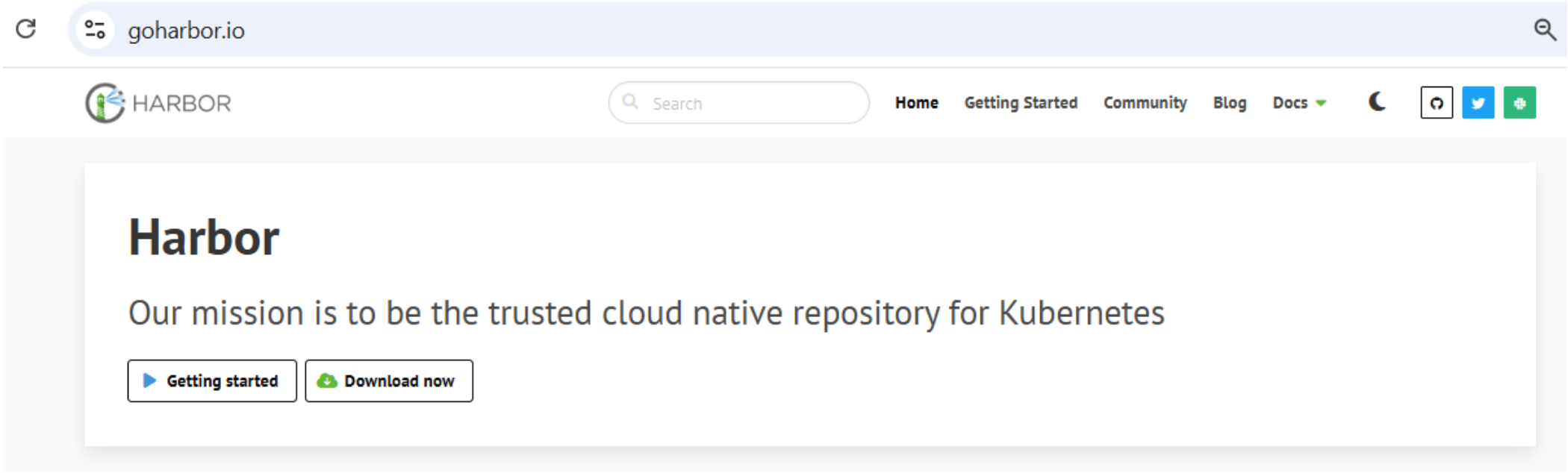
Main features (very short):

- **OCI artifact registry** – store container images, Helm charts, and other OCI artifacts.
- **Security scanning** – detect vulnerabilities using tools like Trivy.
- **Image signing & trust** – verify signed images with Cosign or Notary.
- **RBAC & projects** – multi-tenant access control and quotas.
- **Replication** – sync images across registries (e.g., Docker Hub or other Harbor instances).
- **Policies** – retention rules, immutable tags, security policies.
- **Web UI + API** – manage images, scans, and automation.

Summary: Harbor = **secure, enterprise-grade Docker/OCI registry with governance and replication.**

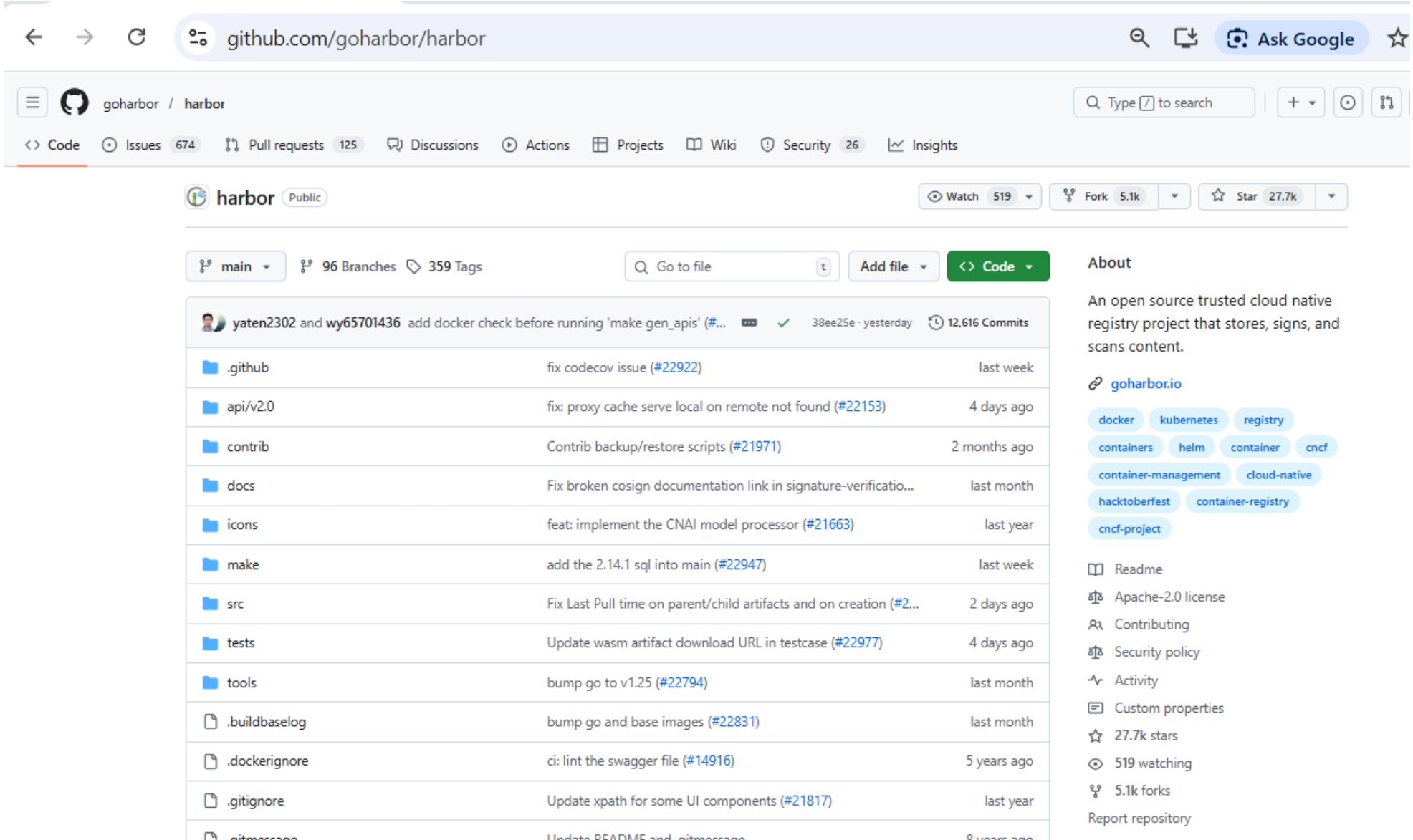
Discover Harbor (web site)

https://https://goharbor.io/

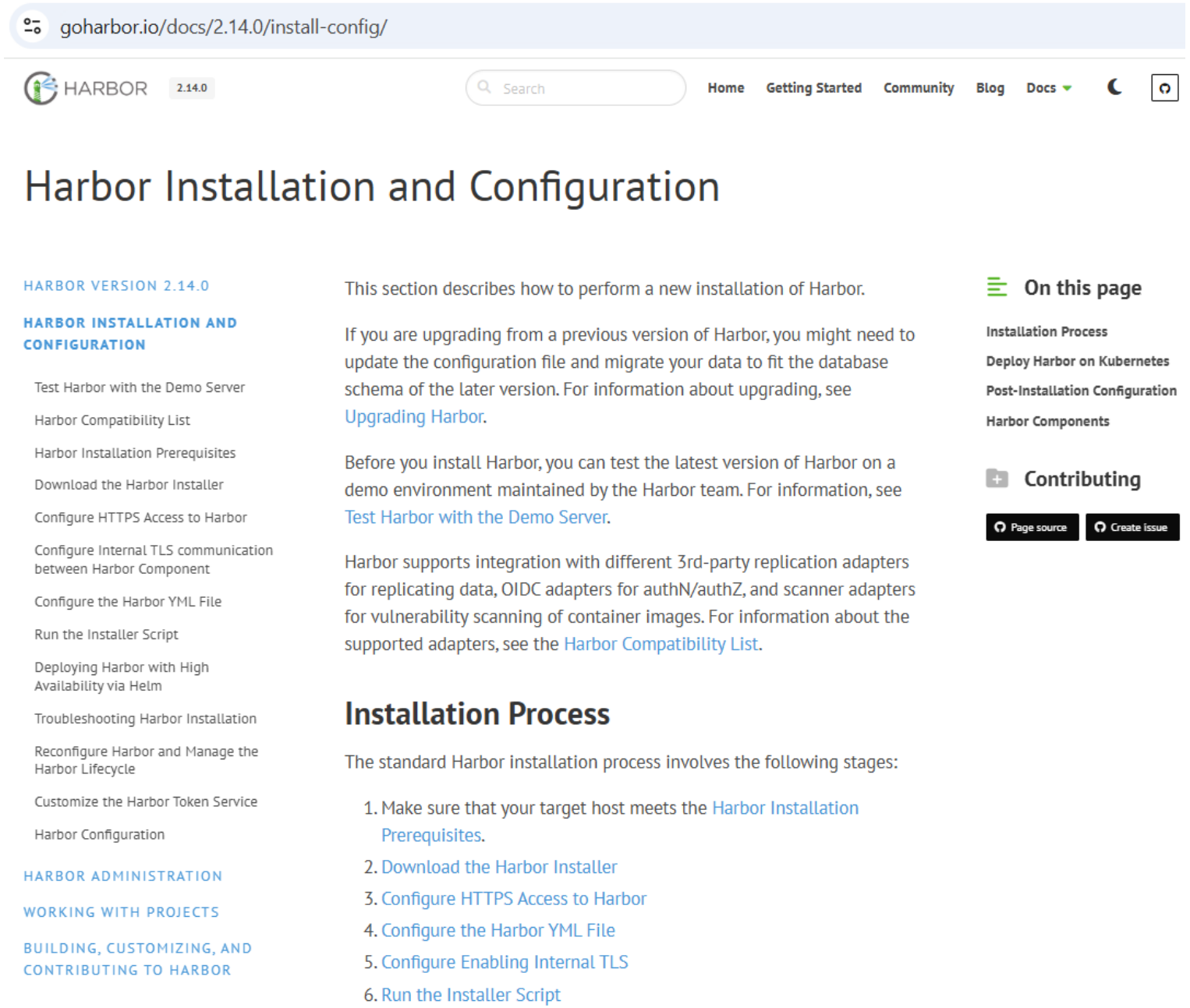


Discover Harbor (source code)

https://github.com/goharbor/harbor



https://https://goharbor.io/ Getting Started



3 officials Ways to install:
1/ Local Offline Installer (=> Docker Compose)
2/ Local Online Installer (=> Docker Compose)
3/ K8s + Helm

This Workshop recommmends alternative 3/: using Helm

We also show 2/ … as failover, in case you are blocked on 3/

Why Installing Harbor?

Because

- It focus on Docker images managment and Vulnerabilities
- It organizes Team read-write permissions for Docker Images pull/push.
- used in production in many companies

- Internally, it is using standard Kubernetes architecture

- It offers easy/standard deployment solution for kubernetes, with Helm

- It is a very complex app, not a « Hello world »
  You can not read/understand all… this is on-purpose !
- You will struggle at deploying it… this is on-purpose !
  A good occasion to learn internals of Kubernetes !

Spoiler Alert: Modern Install = easy commands in 3 lines

helm repo add harbor https://helm.goharbor.io

helm repo update

helm install my-harbor harbor/harbor

In theory …. That's All Folks



But In practice …
A Long (difficult) journey for your kubernetes baptism

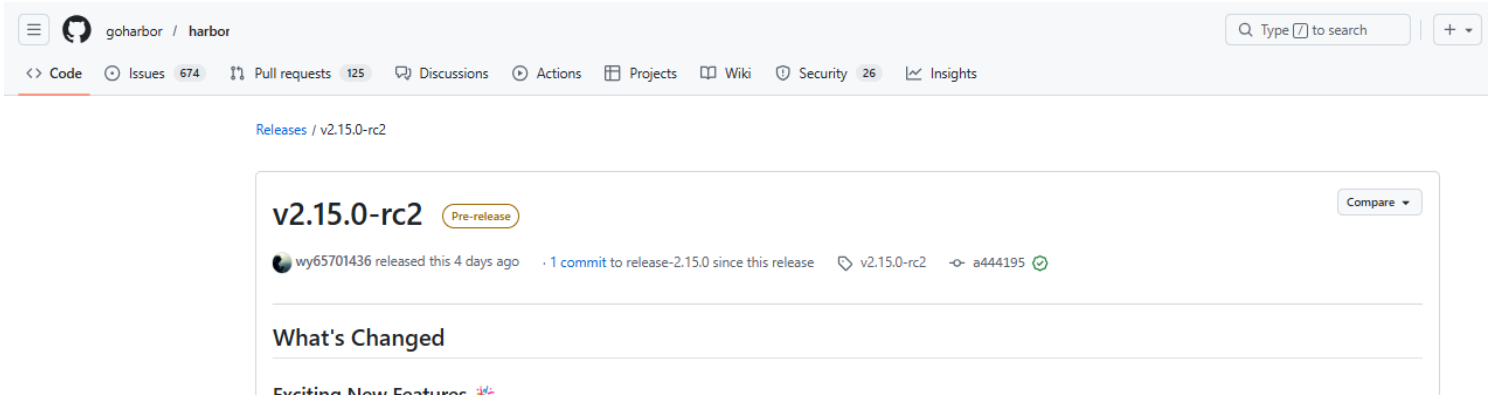
Pre-Requisites = Install Kube cluster + Tools (Helm + Kubectl + K9s)
Then Debug…

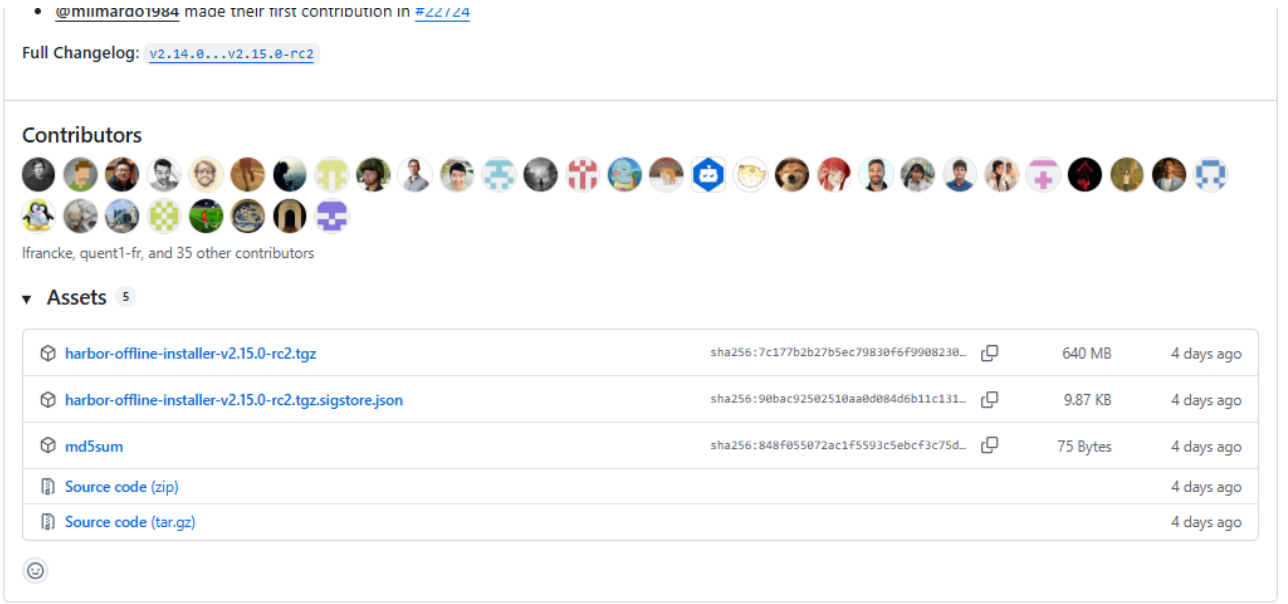
a little detour into the past … Installing the old style

- Download huge tar.gz file
- Unzip
- Copy + edit configuration files
- Generate TLS (https) certificates
- Launch « sudo ./install.sh »
- fix shell scripts
- Discover Result, Debug …

Using Releases - Installer

https://github.com/goharbor/harbor/releases

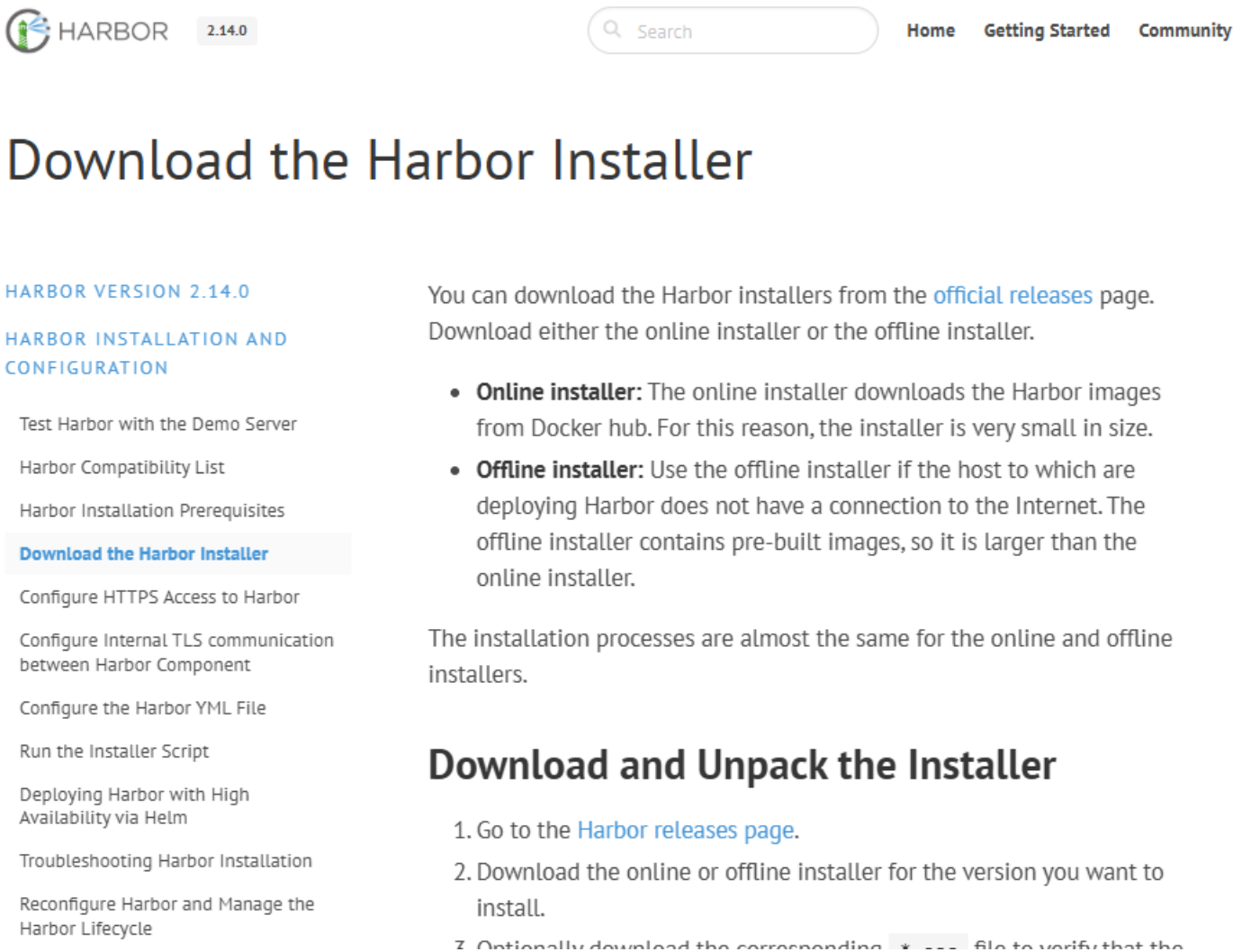




© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Donwload Online/Offline Installer

https://goharbor.io/docs/2.14.0/install-config/download-installer/



Linux/Wsl Install.sh

wsl

wget https://github.com/goharbor/harbor/releases/download/v2.13.5/harbor-online-installer-v2.13.5.tgz

tar zxf harbor*.tgz

cd harbor

cp harbor.yml.tmpl harbor.yml

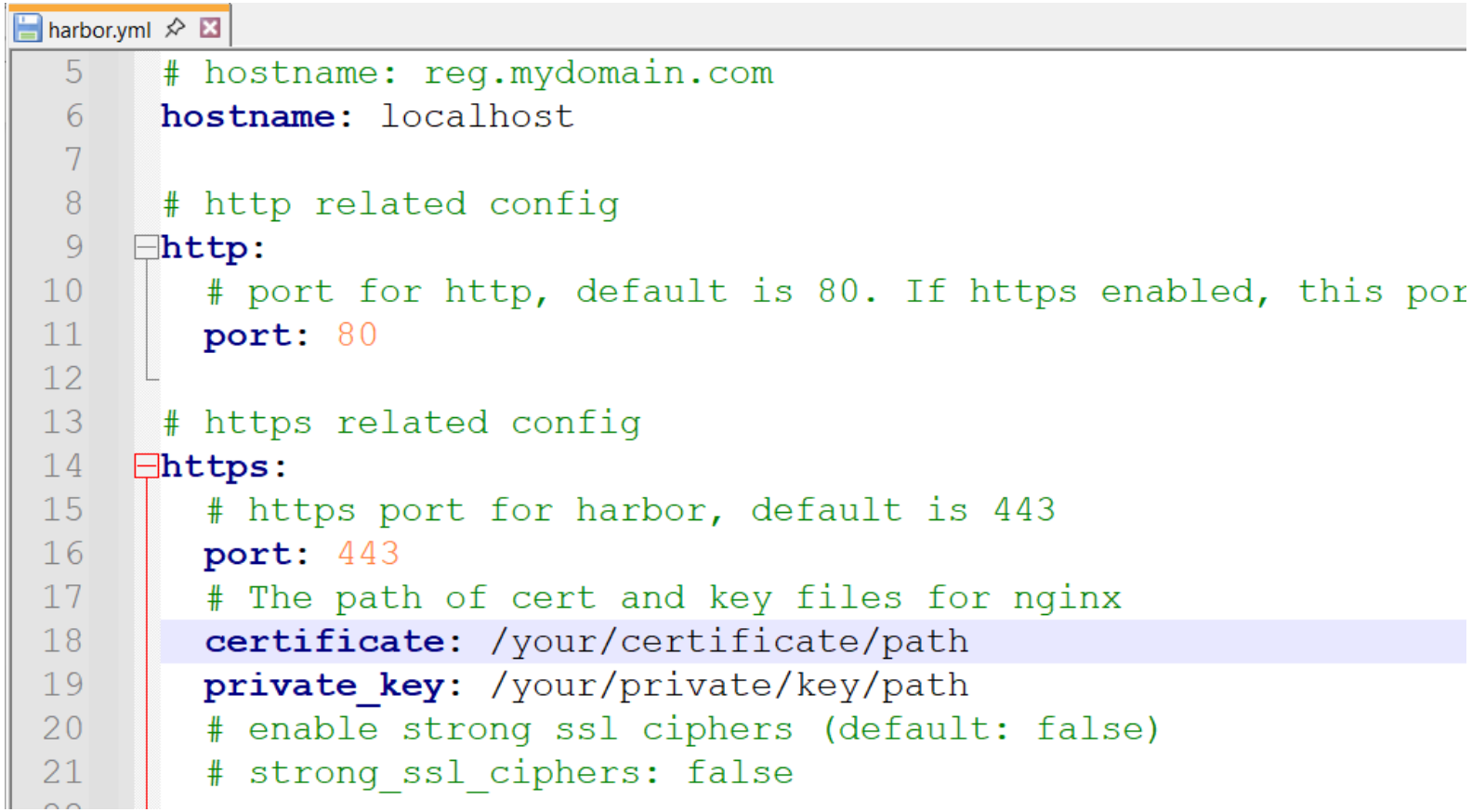
vi harbor.yml

Generate OpenSSL Certificate

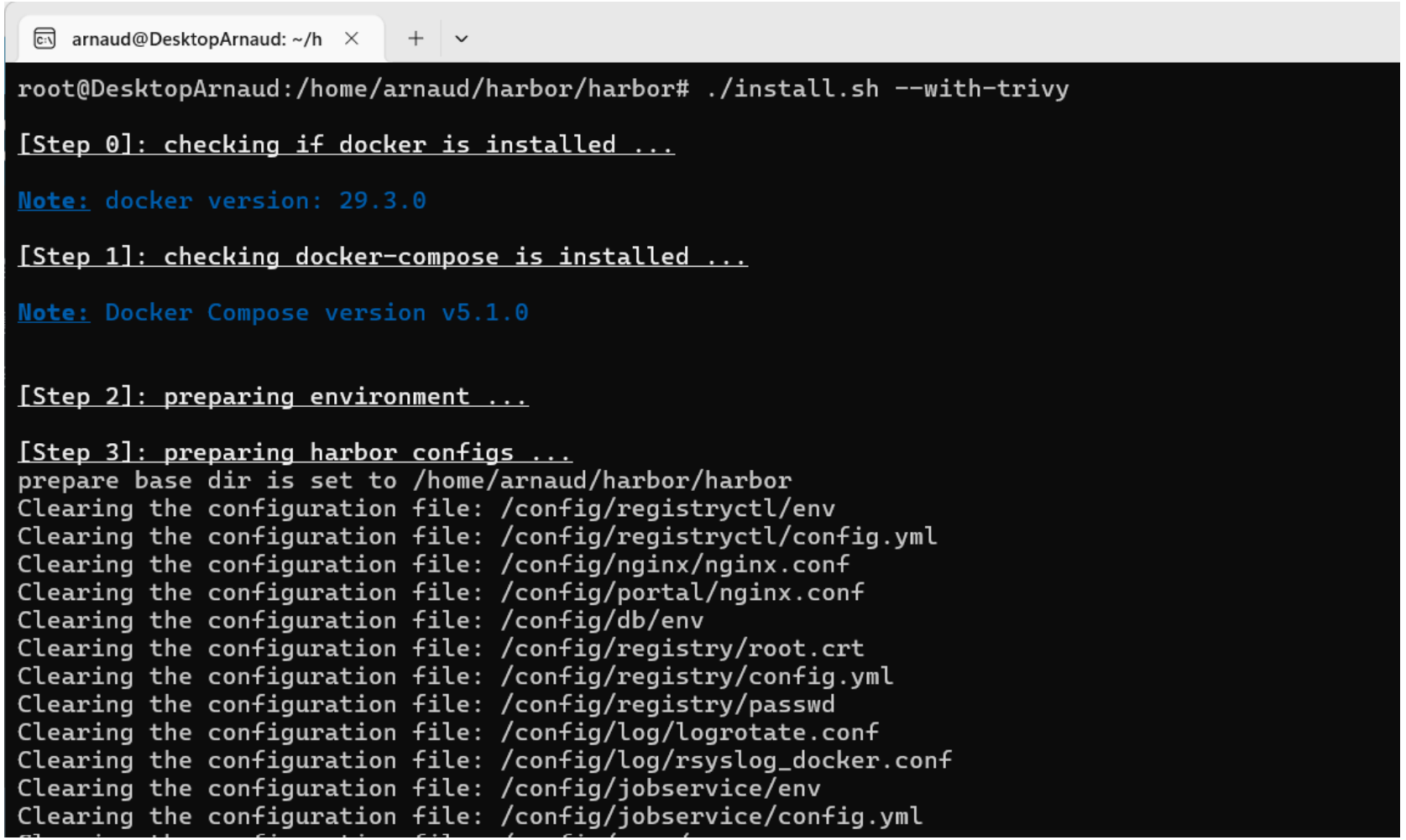
openssl req -x509 -sha256 -days 4000 -nodes -newkey rsa:2048 -subj "/CN=demo.local/C=FR" -keyout rootCA.key -out rootCA.crt

Edit file harbor.yml

hostname: localhost
https:
** certificate: <<YOUR-PATH>>/rootCA.key**
** private_key: <<YOUR-PATH>>/rootCA.crt**



sudo ./install.sh --with-trivy



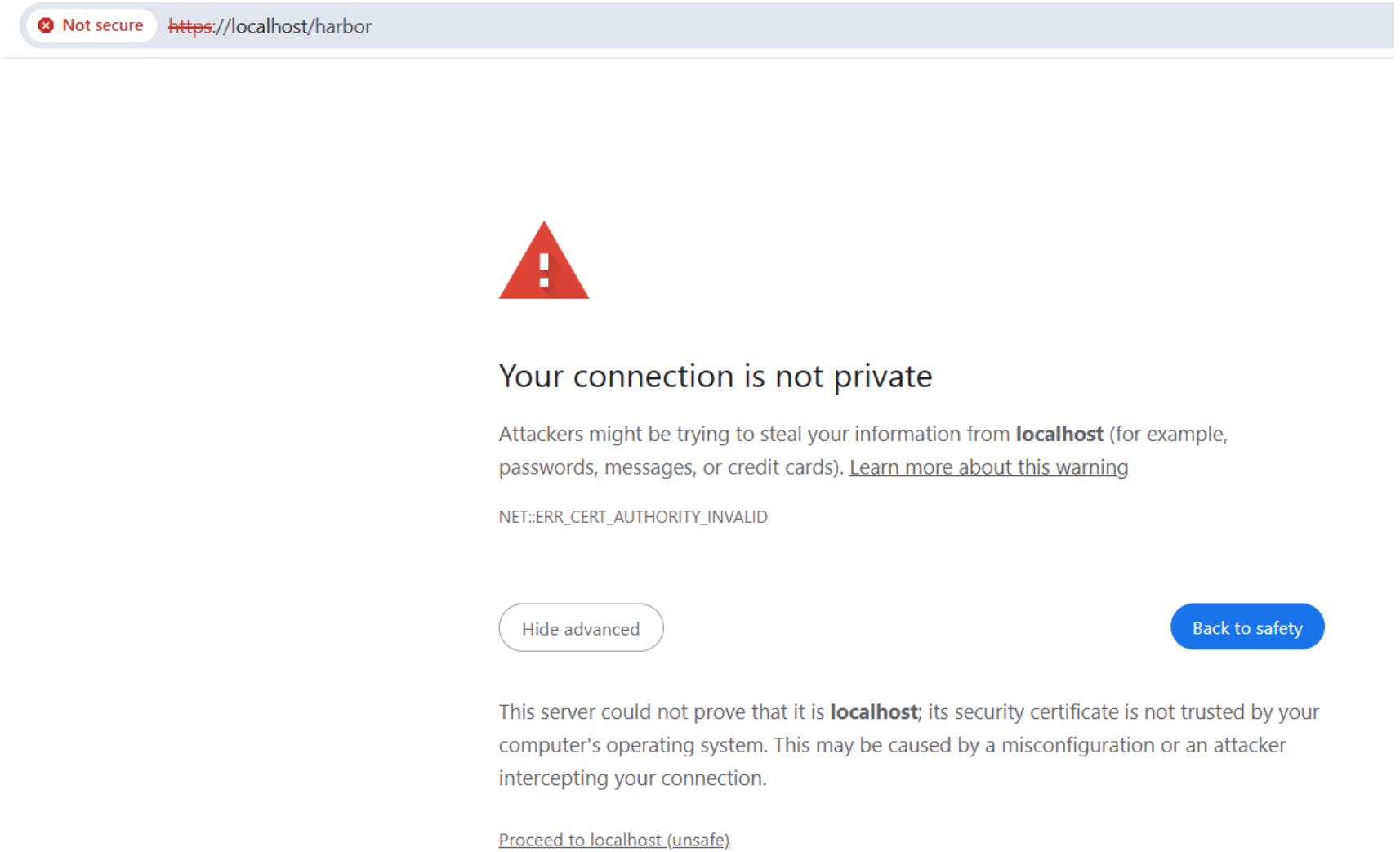
Notice…. Does not work when using your account!!!
Need run as root?!?!! « sudo ./install.sh »

Install => docker compose up -d

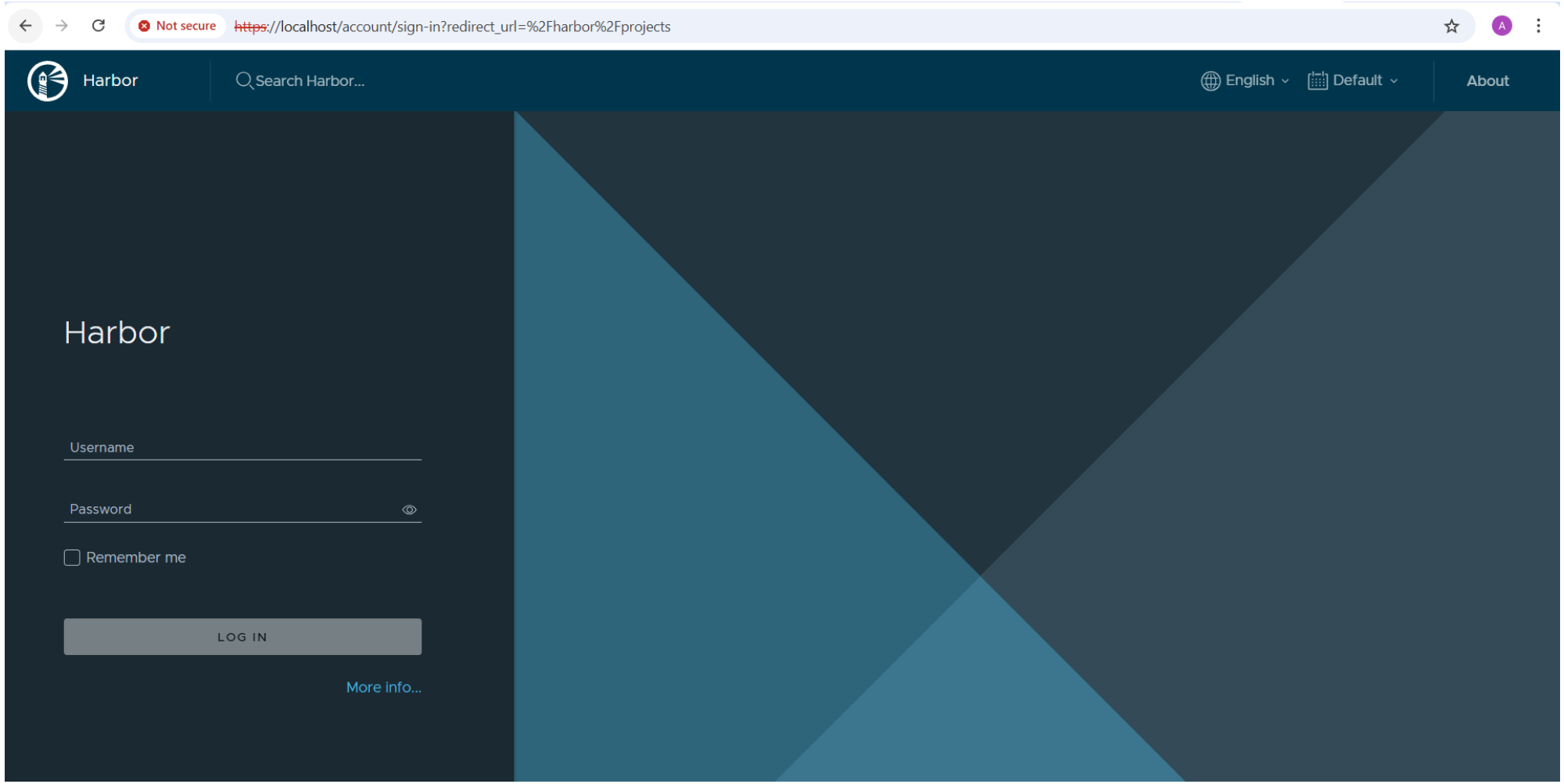
```
[Step 4]: starting Harbor ...
[+] up 18/18
 ✔Image goharbor/trivy-adapter-photon:v2.13.5 Pulled                                59.1s
 ✔Network harbor_harbor                       Created                               0.1s
 ✔Container harbor-log                        Started                               1.4s
 ✔Container registry                          Started                               2.6s
 ✔Container harbor-db                         Started                               2.5s
 ✔Container registryctl                       Started                               2.3s
 ✔Container redis                             Started                               2.2s
 ✔Container harbor-portal                     Started                               2.6s
 ✔Container trivy-adapter                     Started                               3.0s
 ✔Container harbor-core                       Started                               3.2s
 ✔Container harbor-jobservice                 Started                               4.0s
 ✔Container nginx                             Started                               4.0s
✔----Harbor has been installed and started successfully.----
root@DesktopArnaud:/home/arnaud/harbor/harbor#
root@DesktopArnaud:/home/arnaud/harbor/harbor#
root@DesktopArnaud:/home/arnaud/harbor/harbor#
root@DesktopArnaud:/home/arnaud/harbor/harbor# docker compose ps
NAME                IMAGE                                   COMMAND                  SERVICE         CREATED         STATUS                   PORTS
harbor-core         goharbor/harbor-core:v2.13.5            "/harbor/entrypoint.…"   core            3 minutes ago   Up 3 minutes (healthy)
harbor-db           goharbor/harbor-db:v2.13.5              "/docker-entrypoint.…"   postgresql      3 minutes ago   Up 3 minutes (healthy)
harbor-jobservice   goharbor/harbor-jobservice:v2.13.5      "/harbor/entrypoint.…"   jobservice      3 minutes ago   Up 3 minutes (healthy)
harbor-log          goharbor/harbor-log:v2.13.5             "/bin/sh -c /usr/loc…"   log             3 minutes ago   Up 3 minutes (healthy)   127.0.0.1:1
514->10514/tcp
harbor-portal       goharbor/harbor-portal:v2.13.5          "nginx -g 'daemon of…"   portal          3 minutes ago   Up 3 minutes (healthy)
nginx               goharbor/nginx-photon:v2.13.5           "nginx -g 'daemon of…"   proxy           3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:80-
>8080/tcp, [::]:80->8080/tcp, 0.0.0.0:443->8443/tcp, [::]:443->8443/tcp
redis               goharbor/redis-photon:v2.13.5           "redis-server /etc/r…"   redis           3 minutes ago   Up 3 minutes (healthy)
registry            goharbor/registry-photon:v2.13.5        "/home/harbor/entryp…"   registry        3 minutes ago   Up 3 minutes (healthy)
registryctl         goharbor/harbor-registryctl:v2.13.5     "/home/harbor/start.…"   registryctl     3 minutes ago   Up 3 minutes (healthy)
trivy-adapter       goharbor/trivy-adapter-photon:v2.13.5   "/home/scanner/entry…"   trivy-adapter   3 minutes ago   Up 3 minutes (healthy)
root@DesktopArnaud:/home/arnaud/harbor/harbor#
```

https://localhost:443/harbor

Proceed Unsafe (self-signed certificate)



Login: admin / Harbor12345

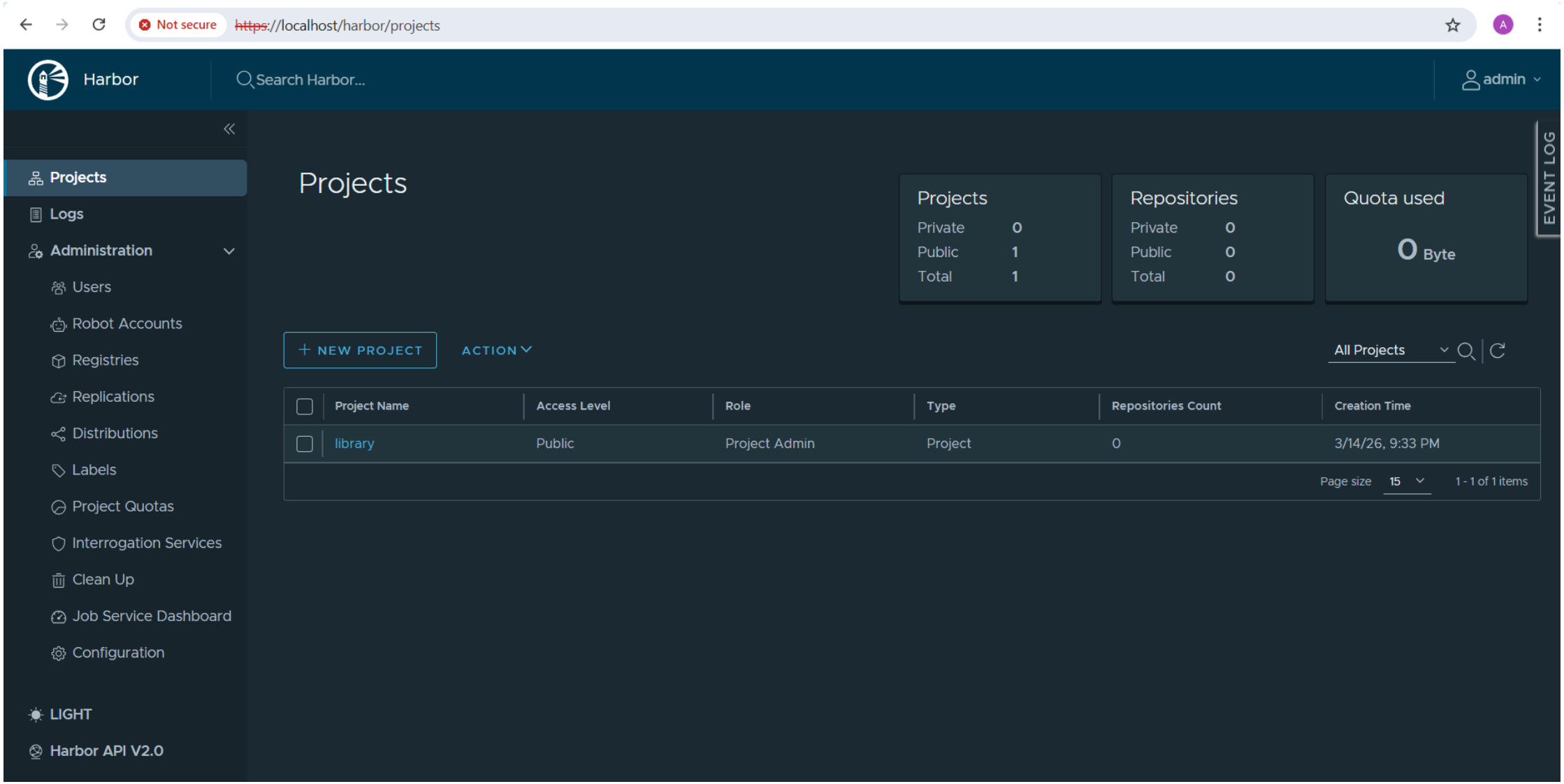


See Config « admin » / « Harbor12345 »

harbor.yml

```
44
45  # The initial password of Harbor admin
46  # It only works in first time to install harbor
47  # Remember Change the admin password from UI after launching Harbor.
48  harbor_admin_password: Harbor12345
```

It Works !


Not secure
https://localhost/harbor/projects
Harbor
Search Harbor...
admin
Projects
Logs
Administration
Users
Robot Accounts
Registries
Replications
Distributions
Labels
Project Quotas
Interrogation Services
Clean Up
Job Service Dashboard
Configuration
LIGHT
Harbor API V2.0
Projects
Projects
Private 0
Public 1
Total 1
Repositories
Private 0
Public 0
Total 0
Quota used
0 Byte
EVENT LOG
+ NEW PROJECT
ACTION
All Projects
Project Name
Access Level
Role
Type
Repositories Count
Creation Time
library
Public
Project Admin
Project
0
3/14/26, 9:33 PM
Page size 15
1 - 1 of 1 items

But How Deceptive !!

only plain-old legacy Shell + « Docker-Compose »

unmanaged solution, that nobody uses in real PROD !!
(no cluster / not distributed / not scalable
/ no monitoring / no auto-restart / no auto-scaling up&down / …)

And nothing new, no Kubernetes !

Better Install
Using Kubernetes + Helm

using Linux | Windows WSL > Kind
… with teminal Shell command lines
(see also next alternative, using Podman GUI)

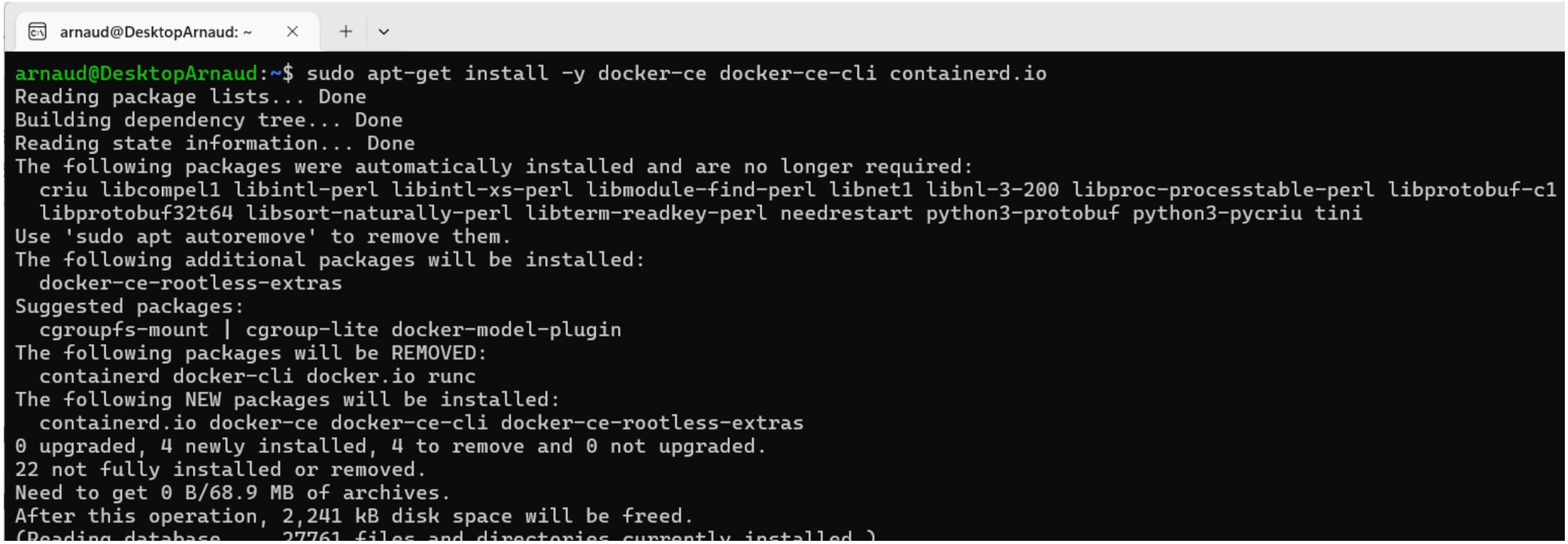
Linux vs « WSL » … Kubernetes « Kind » ?

Docker run almost « natively » under Linux
On windows => either use
- « WSL » = Windows Sub-system for Linux (=> like SSH + real Linux VM)
- DockerDesktop for Windows… paid solution, NOT used much in companies
- Podman

Kubernetes can run « natively » under linux, but may be difficult to install
example of packaged solutions: « minikube », « k3s », « kind », …
« Kind » = « Kubernetes In Docker »

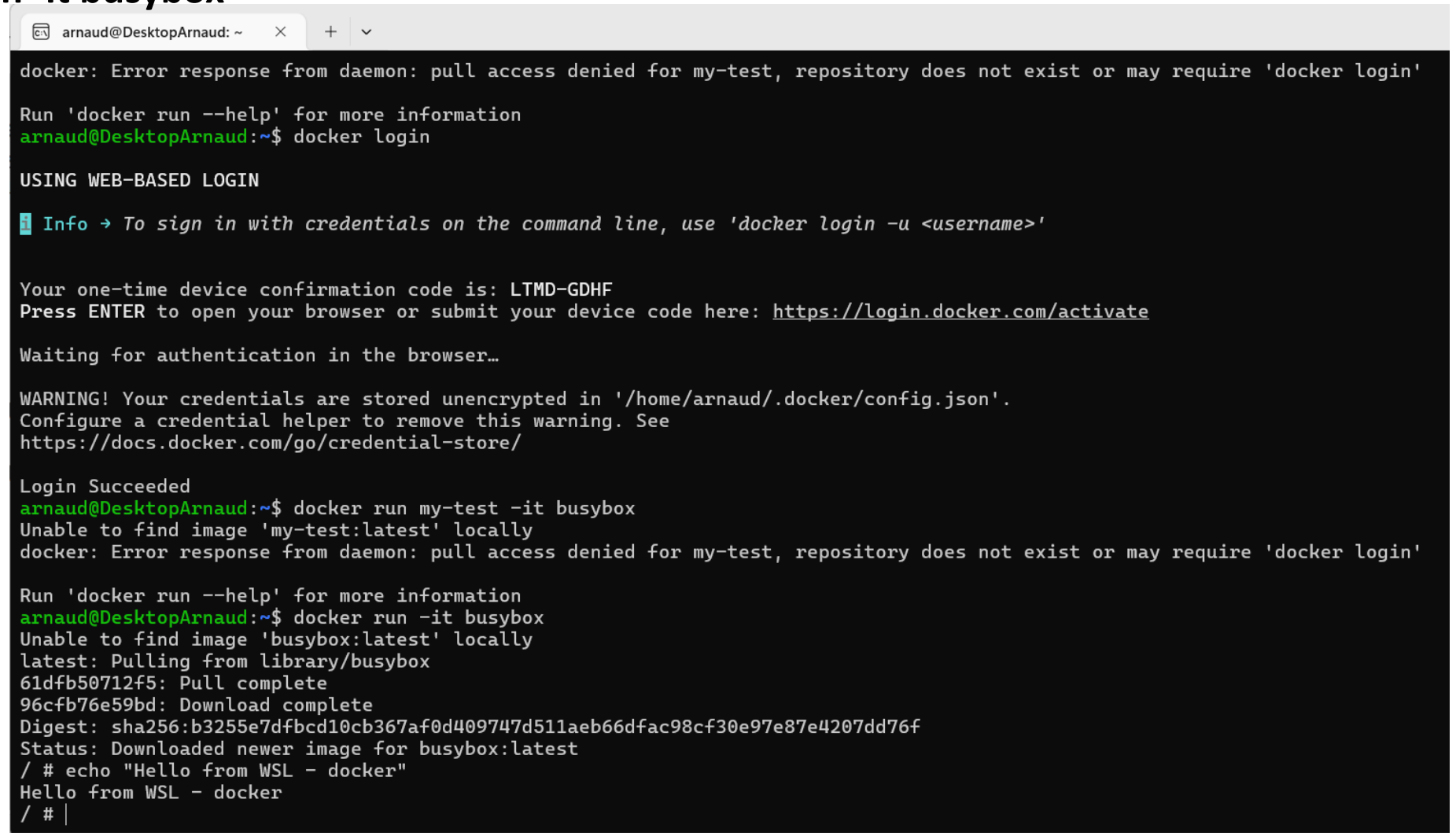
Windows > WSL > Docker
Linux > Docker

sudo apt install -y docker-ce docker-ce-cli containerd.io



WSL > test Docker

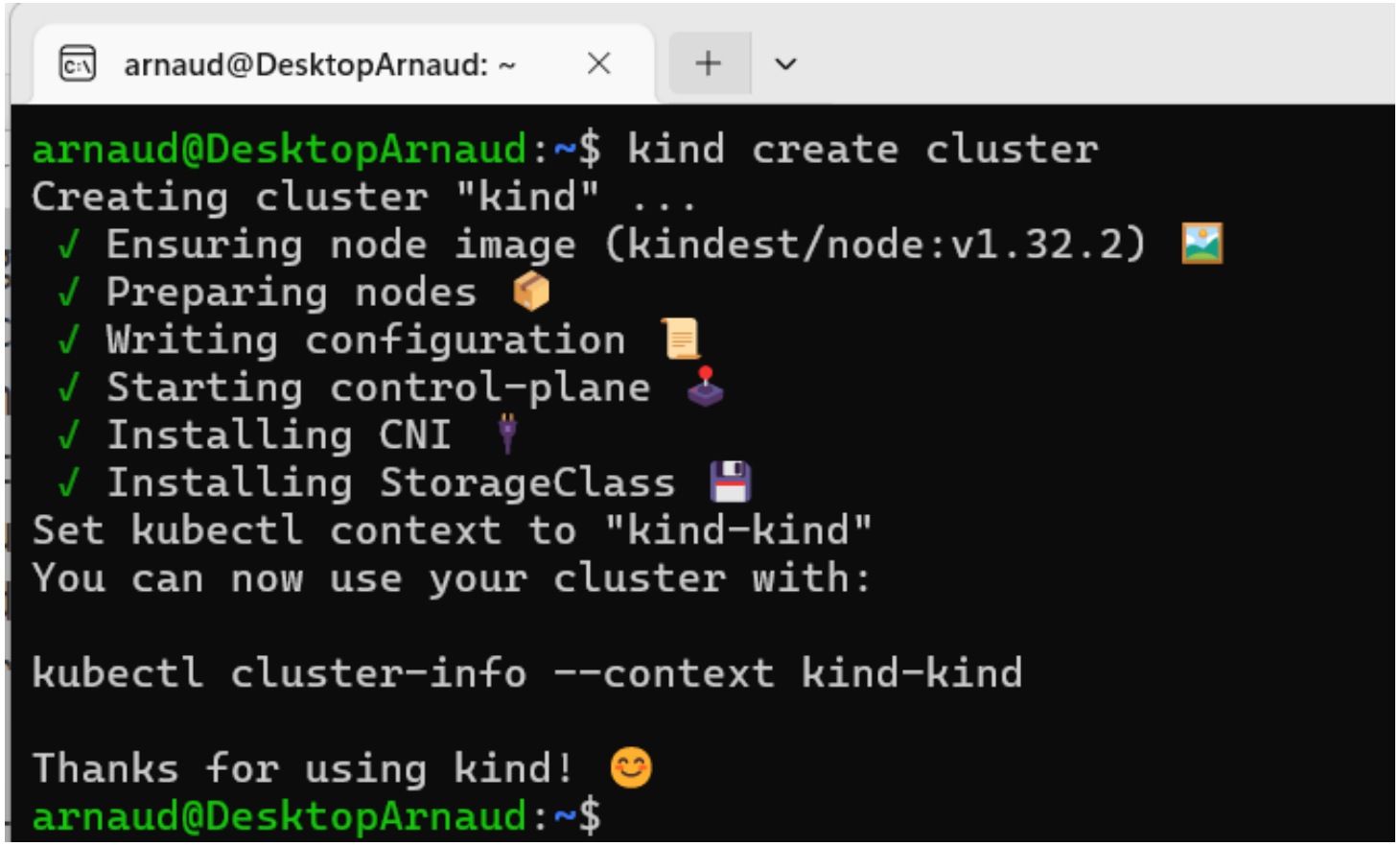
docker ps -a
docker login
docker run -it busybox



(WSL | Linux) > sudo apt-get install kind kubectl kind create cluster

sudo apt-get install kind kubectl

kind create cluster

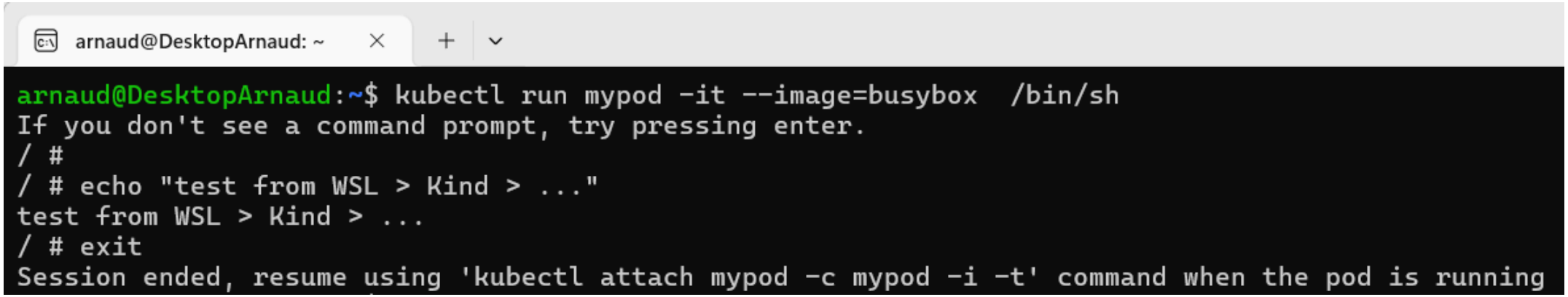


Test Kubectl

kubectl config get-contexts

kubectl get pod

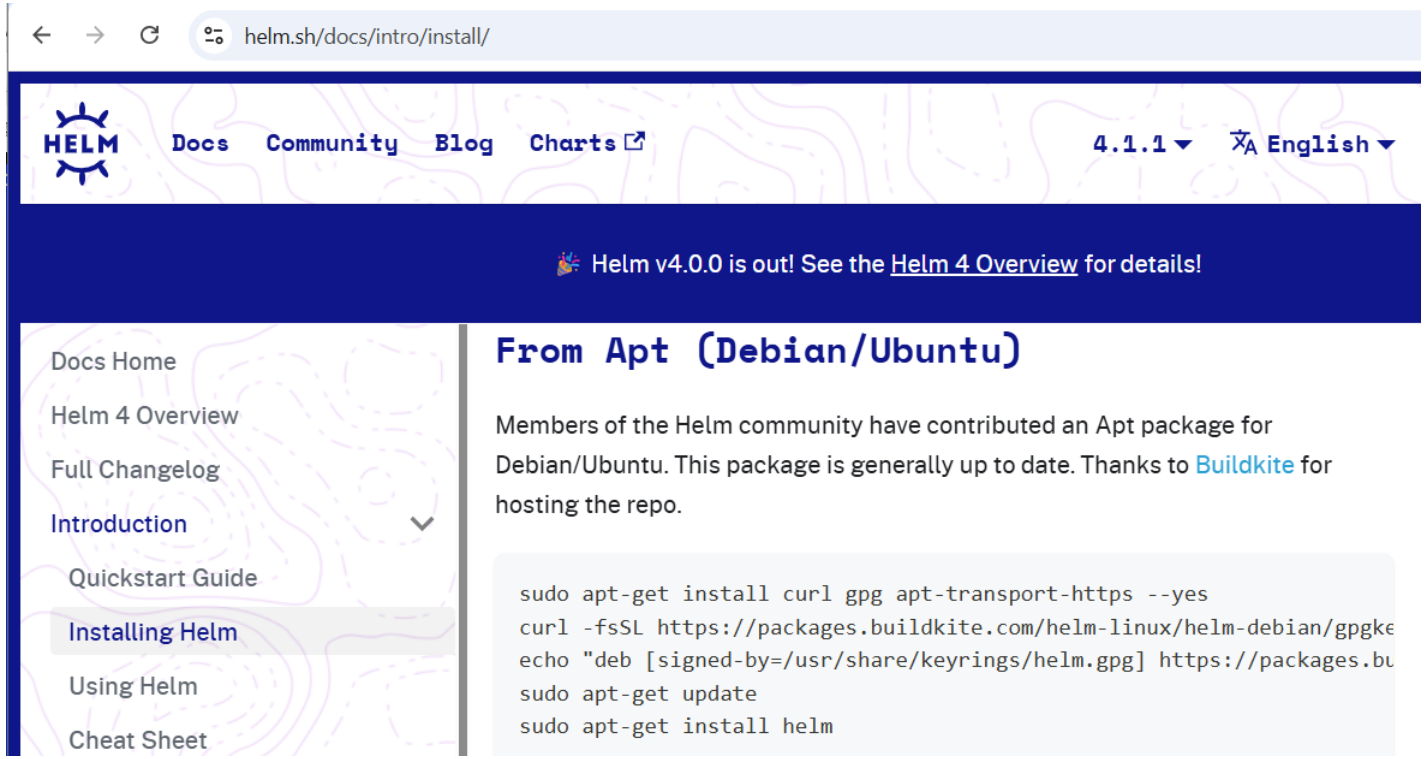
kubectl run mypod -it --image=busybox /bin/sh
kubectl delete pod mypod



Install Helm

https://helm.sh/docs/intro/install/

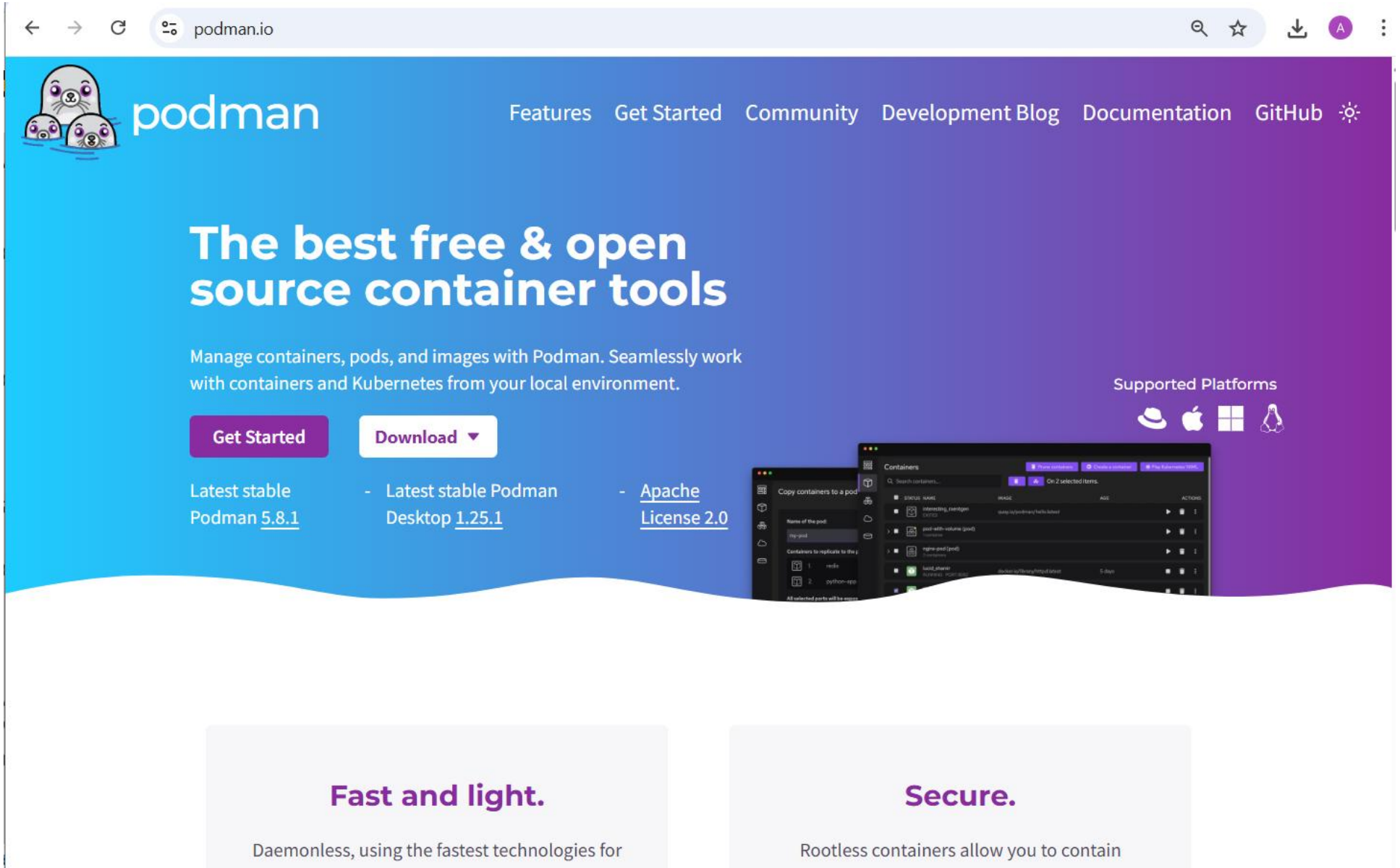
sudo apt-get install curl gpg apt-transport-https --yes
curl -fsSL https://packages.buildkite.com/helm-linux/helm-debian/gpgkey | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/helm.gpg] https://packages.buildkite.com/helm-linux/helm-debian/any/ any main" | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list
sudo apt-get update
sudo apt-get install helm



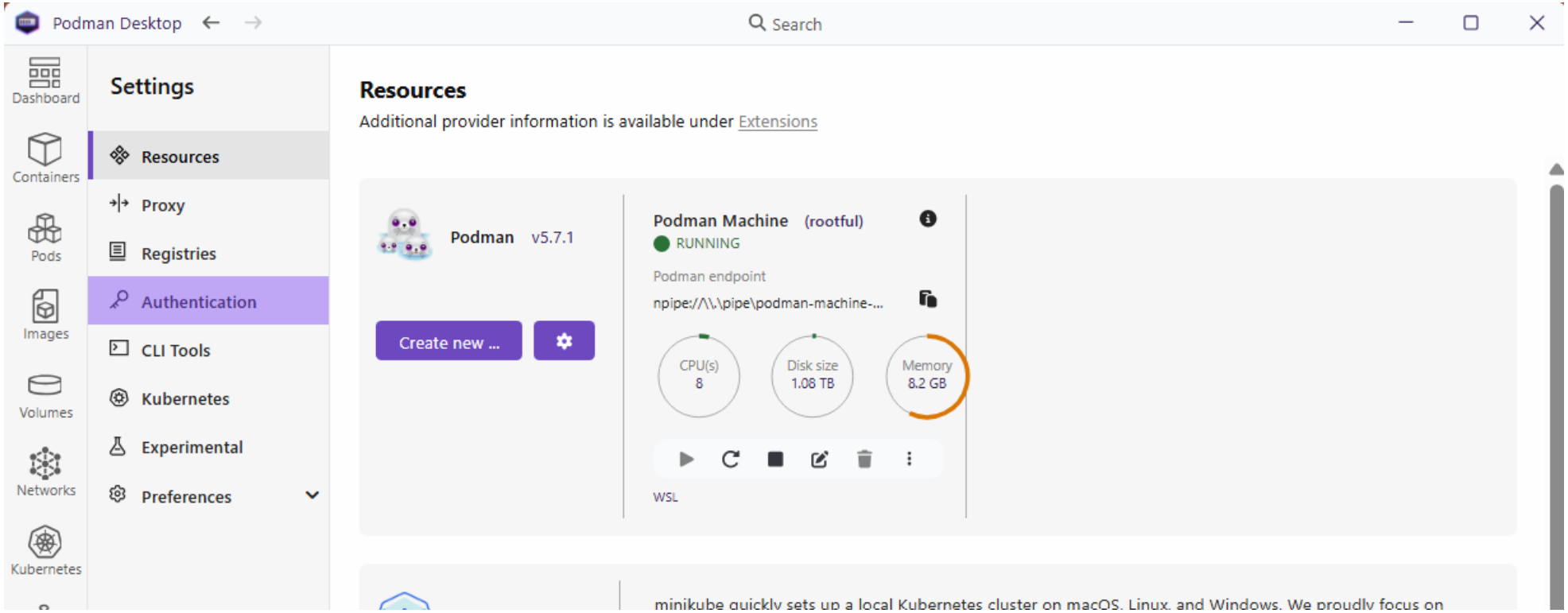
Alternative for WSL (+ Kind)
… Using Podman (+ Kind)

Install Podman

https://podman.io



Podman > Create « Podman Machine »



alias docker=podman

« podman.exe » is a simple executable file

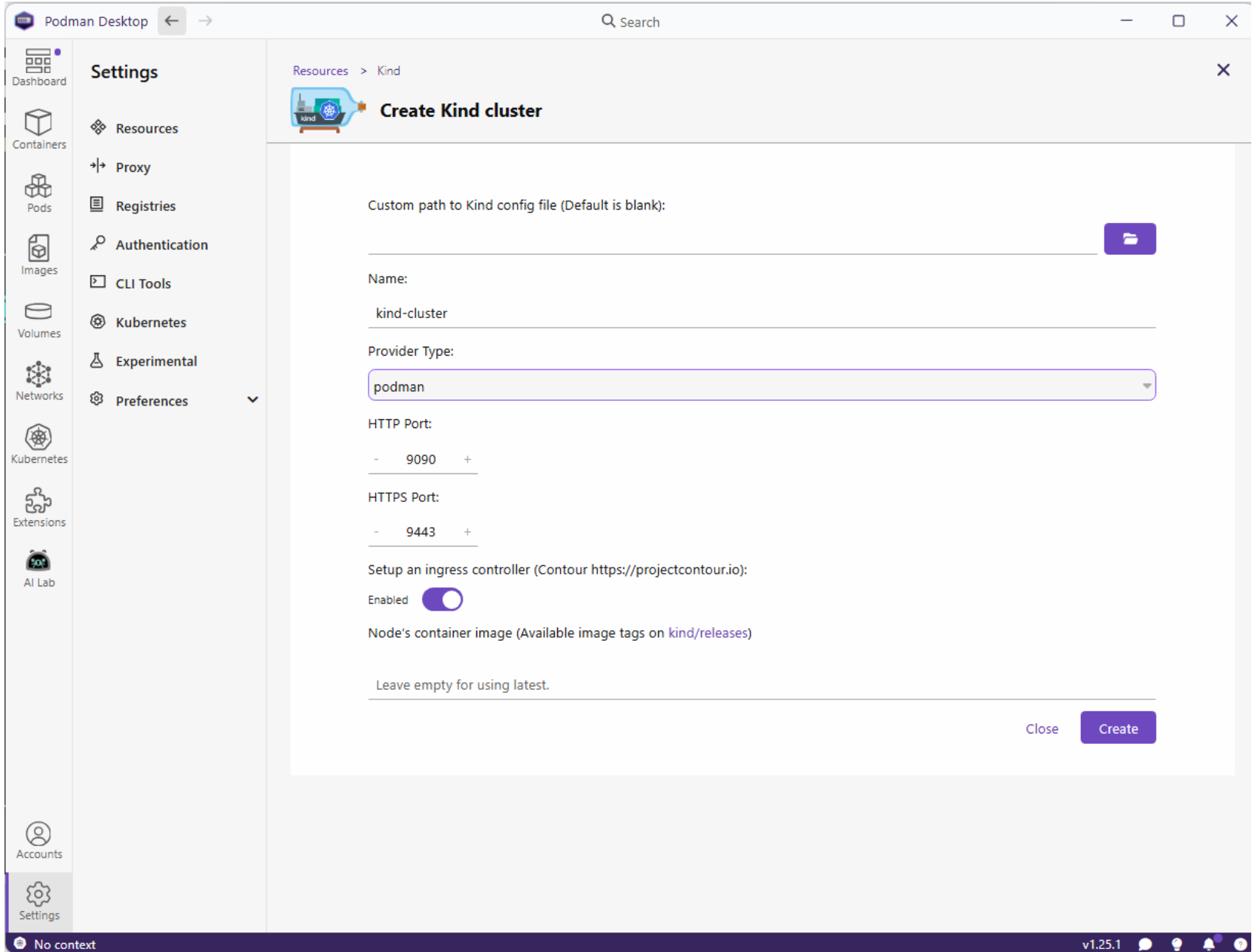
… that is backward compatible with ~all Docker commands

In your ${HOME}/.profile or ${HOME}.bashr, you can even use

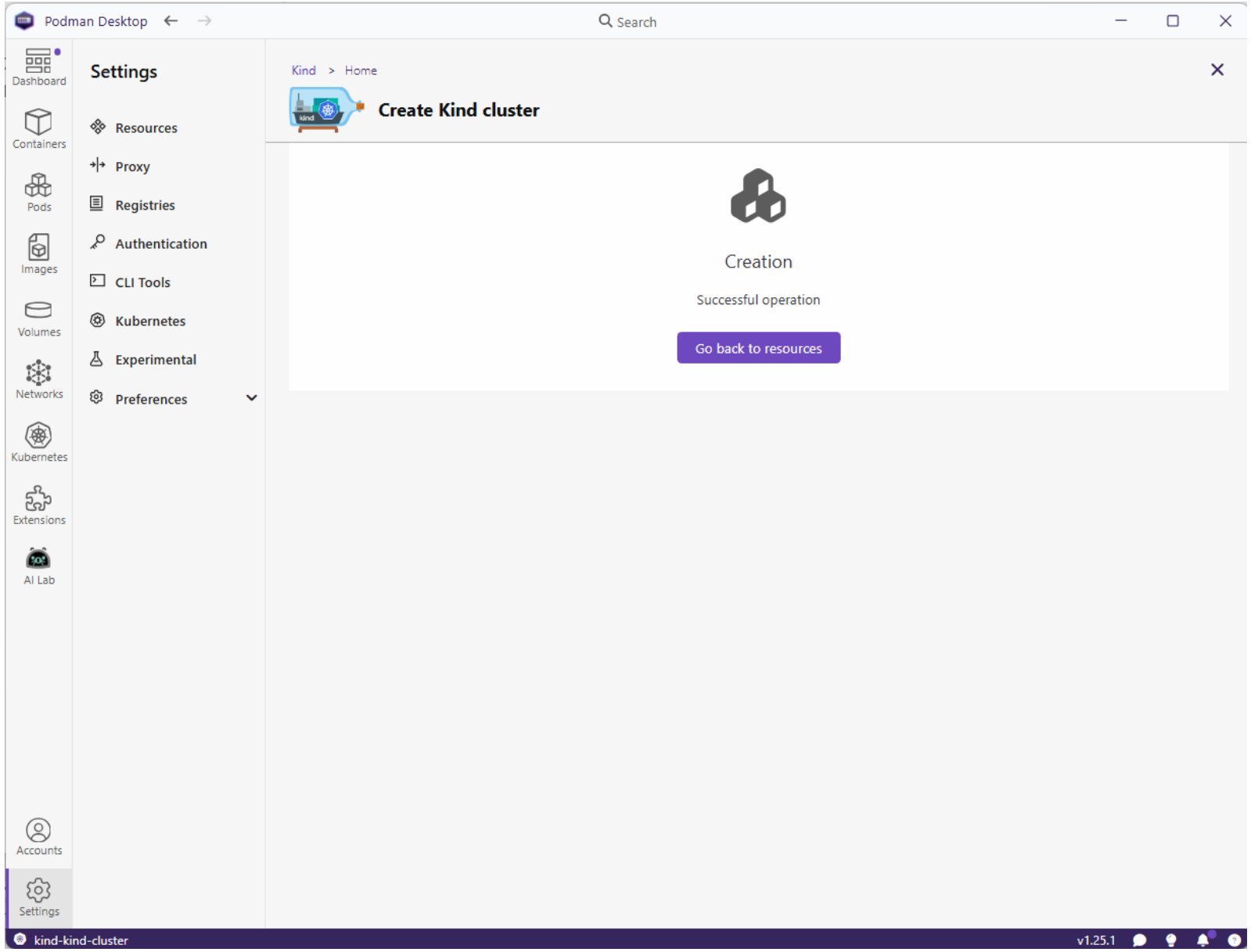
alias docker=podman

podman ps		**docker ps**
podman run …	=	**docker run …**
podman kill ${container}		**docker kill ${container}**
…		**…**

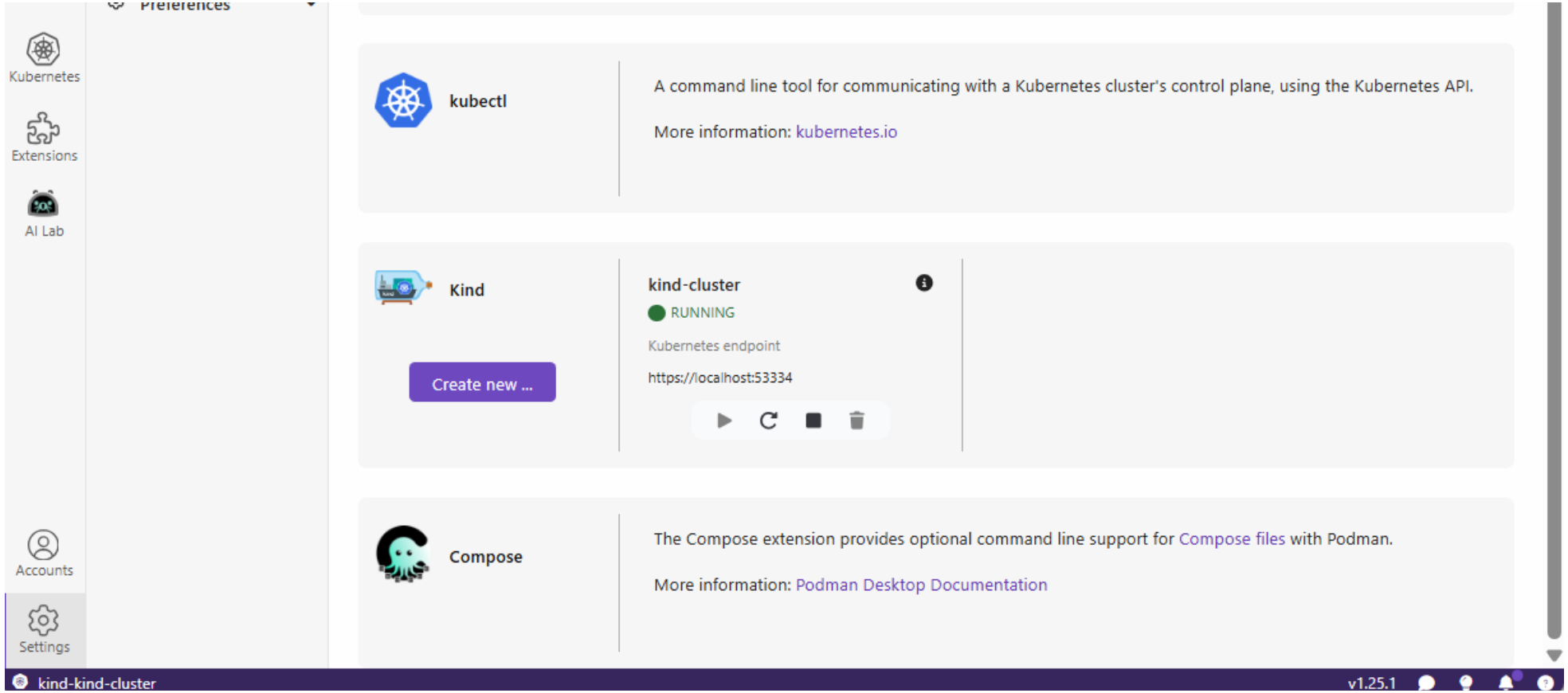
Step 2: Podman > Create Kind cluster



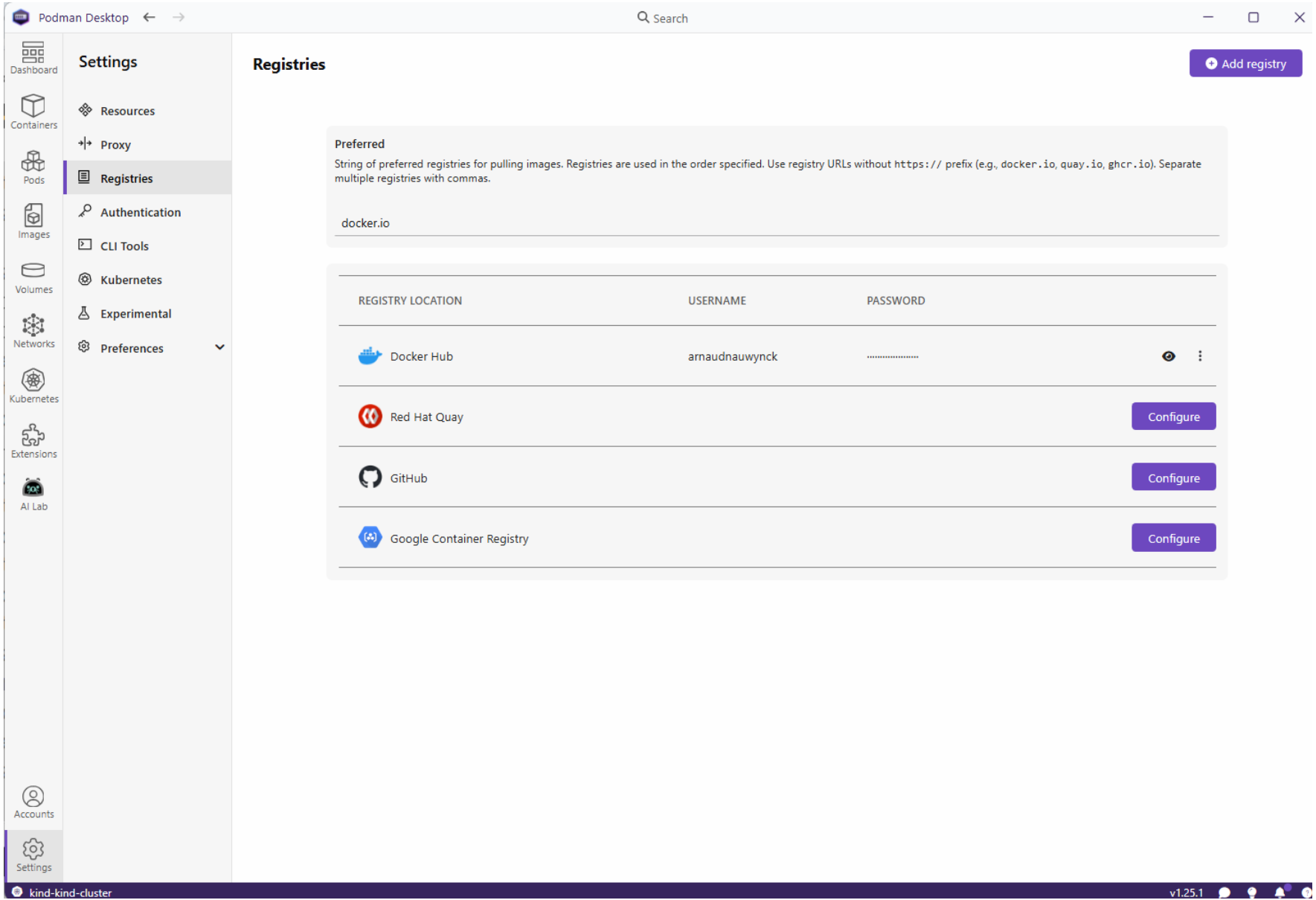
Wait ~3 minutes



Podman Settings > Kind > RUNNING



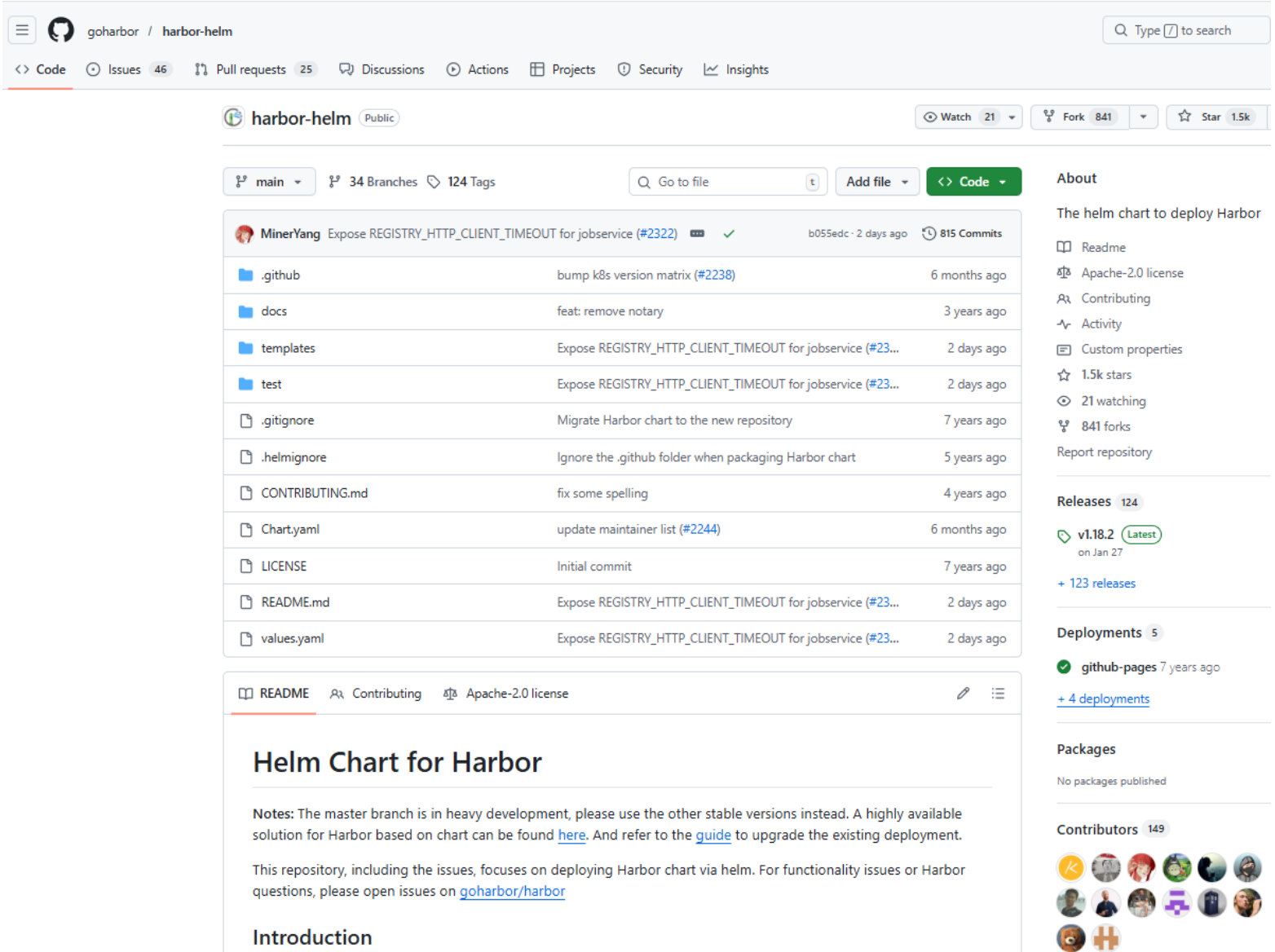
Configure podman with your Dockerhub account



Installing Harbor using K8s + Helm

Helm Chart for Harbor

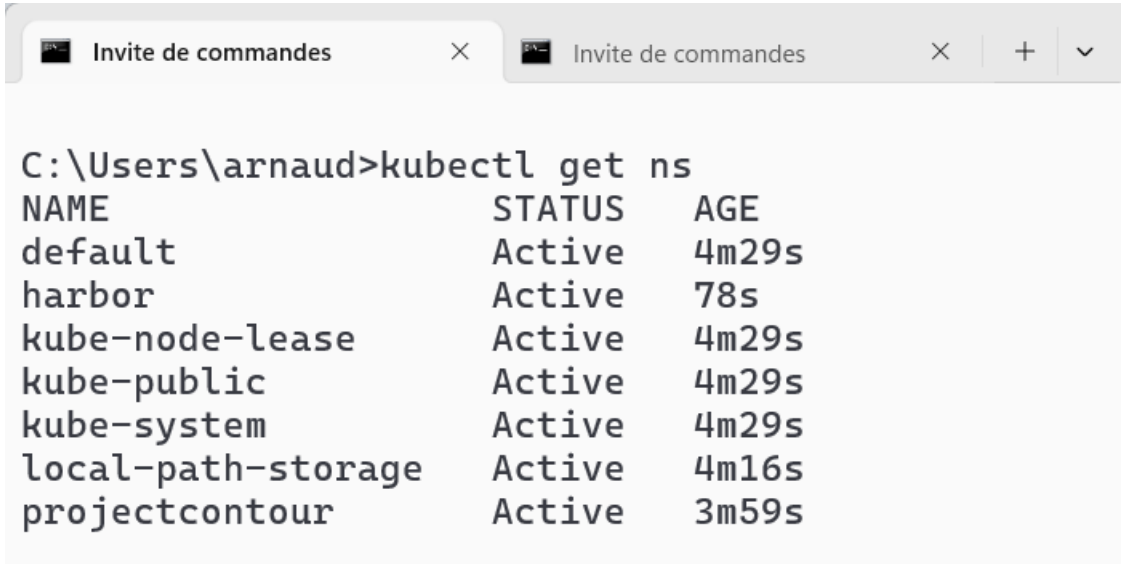
https://github.com/goharbor/harbor-helm



(optional) Kubectl create namespace

kubectl create ns harbor

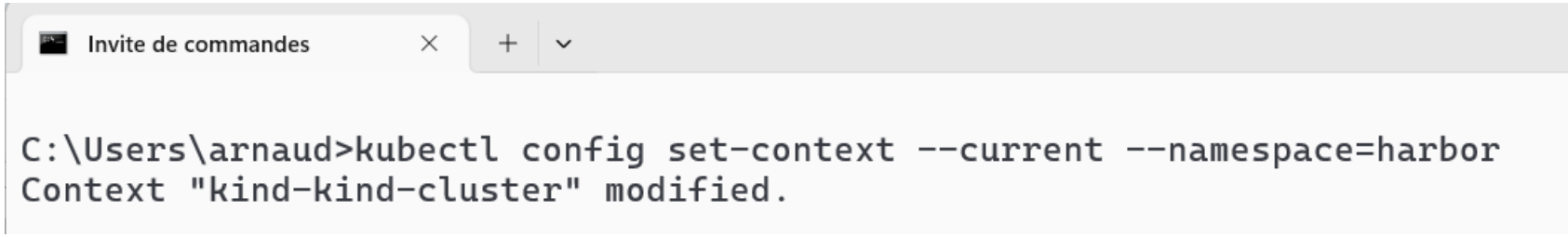
kubectl get ns



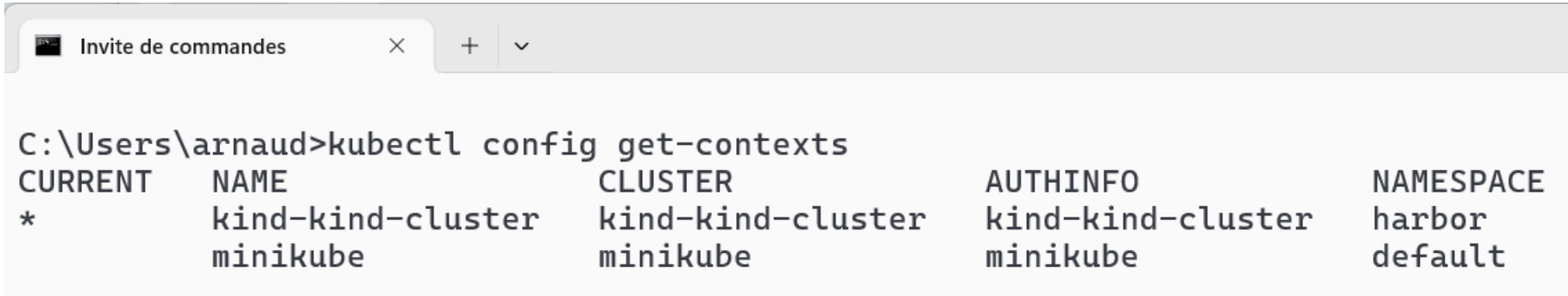
(optional) Kubeconfig… set default namespace

Every kubectl commands must add option « **-n harbor** » to set namespace !

kubectl config set-context --current --namespace=harbor



kubectl config get-contexts



in Terminal: helm repo add

helm repo add harbor https://helm.goharbor.io
helm repo update

helm repo list

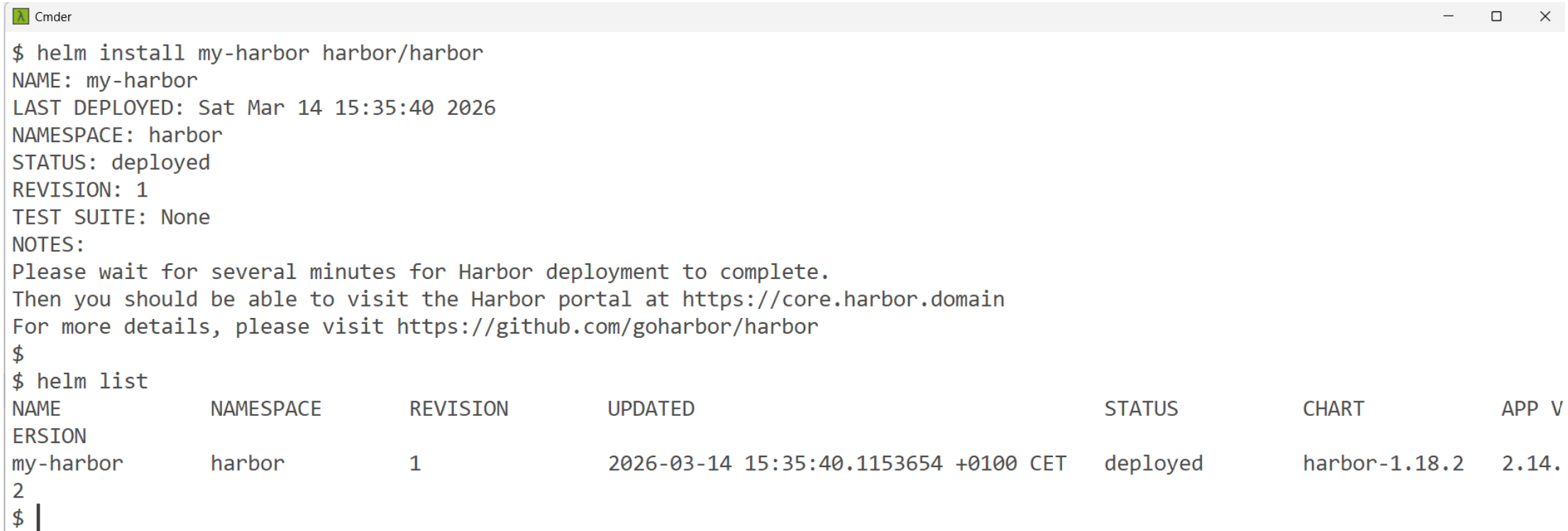
Cmder

```
$  helm repo add harbor https://helm.goharbor.io
"harbor" has been added to your repositories
$
$ helm repo list
NAME            URL
spark-operator  https://kubeflow.github.io/spark-operator
harbor          https://helm.goharbor.io
$
$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "spark-operator" chart repository
...Successfully got an update from the "harbor" chart repository
Update Complete. ⎈Happy Helming!⎈
$
```

Helm install

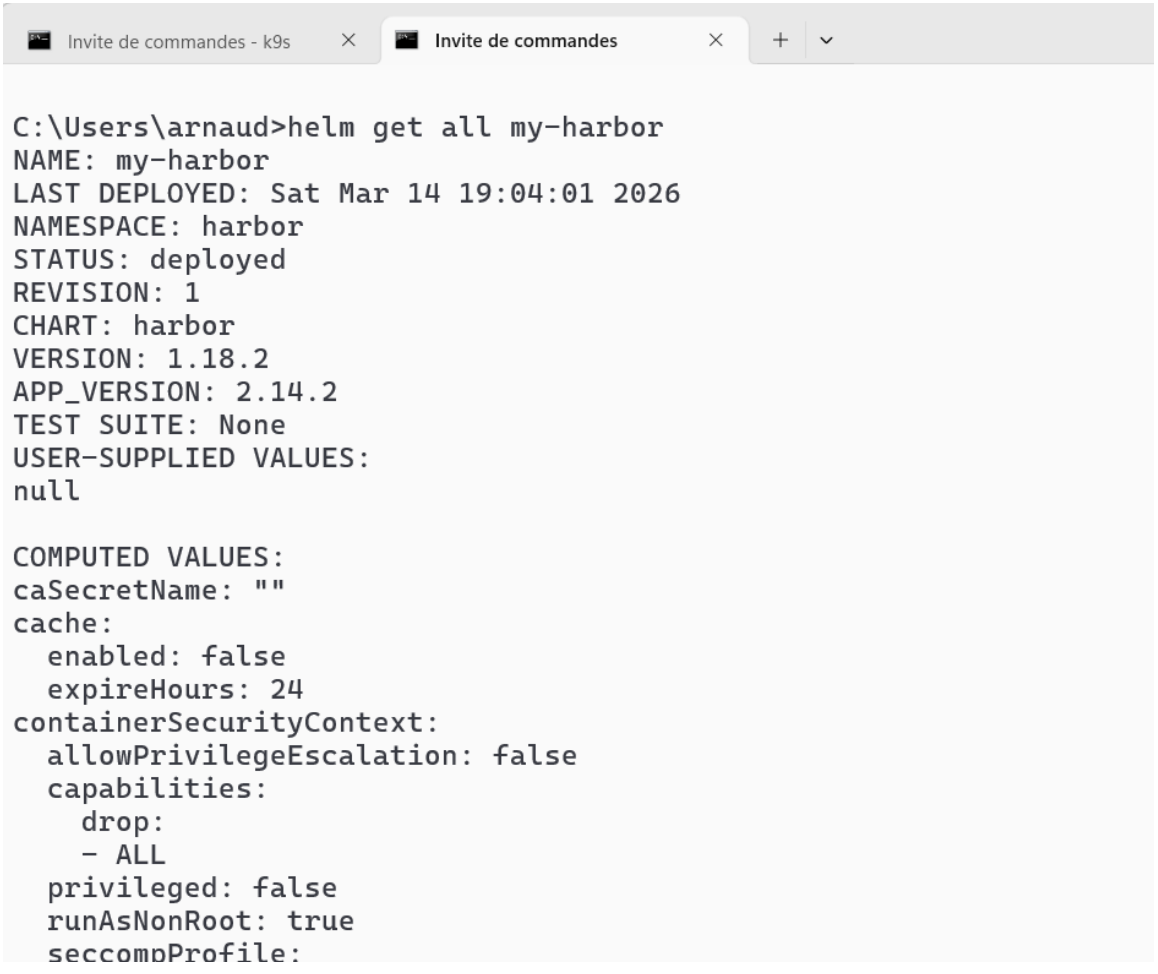
helm install my-harbor harbor/harbor

helm list



Helm get all (helm install --dry-run)

helm get all my-harbor > my-harbor.yaml



Kubectl … WAIT ~5 minutes (Image Pull)….

kubectl get all

kubectl get pod

… Wait pod « ContainerCreating »

kubectl get deployment

… Wait READY 0/1

kubectl get event -w

Check… kubectl get all

kubectl get all

Cmder

```
$ kubectl get all
NAME                                      READY   STATUS              RESTARTS   AGE
pod/my-harbor-core-6cdbbd5f6c-djbpw       0/1     Terminating         0          3m10s
pod/my-harbor-core-7ff8d69f85-wffzx       0/1     ContainerCreating   0          80s
pod/my-harbor-jobservice-685b6dc444-jbk7g 0/1     ContainerCreating   0          80s
pod/my-harbor-portal-f8f6c556f-km2tt      0/1     Terminating         0          3m10s
pod/my-harbor-portal-f8f6c556f-trd48      0/1     ContainerCreating   0          80s
pod/my-harbor-registry-77c5855bcf-t4tzf   0/2     ContainerCreating   0          80s
pod/my-harbor-registry-ccb5fdf6c-tsvxg    0/2     Terminating         0          3m10s
pod/my-harbor-trivy-0                     0/1     Terminating         0          3m8s

NAME                          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)             AGE
service/my-harbor-core        ClusterIP   10.96.154.0     <none>        80/TCP              81s
service/my-harbor-database    ClusterIP   10.96.146.211   <none>        5432/TCP            81s
service/my-harbor-jobservice  ClusterIP   10.96.184.127   <none>        80/TCP              81s
service/my-harbor-portal      ClusterIP   10.96.3.23      <none>        80/TCP              81s
service/my-harbor-redis       ClusterIP   10.96.205.11    <none>        6379/TCP            81s
service/my-harbor-registry    ClusterIP   10.96.92.2      <none>        5000/TCP,8080/TCP   80s
service/my-harbor-trivy       ClusterIP   10.96.77.237    <none>        8080/TCP            80s

NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/my-harbor-core       0/1     1            0           80s
```

bash.exe

… WAIT ~5 minutes …

kubectl get pod
kubectl get deployment

Cmder

```
$ kubectl get pod
NAME                                  READY   STATUS              RESTARTS   AGE
my-harbor-core-7ff8d69f85-wffzx       0/1     ContainerCreating   0          2m51s
my-harbor-database-0                  0/1     Init:0/1            0          89s
my-harbor-jobservice-685b6dc444-jbk7g 0/1     ContainerCreating   0          2m51s
my-harbor-portal-f8f6c556f-trd48      0/1     ContainerCreating   0          2m51s
my-harbor-redis-0                     0/1     ContainerCreating   0          89s
my-harbor-registry-77c5855bcf-t4tzf   0/2     ContainerCreating   0          2m51s
my-harbor-registry-ccb5fdf6c-tsvxg    0/2     Terminating         0          4m41s
my-harbor-trivy-0                     0/1     ContainerCreating   0          7s
$ kubectl get deployment
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
my-harbor-core       0/1     1            0           3m7s
my-harbor-jobservice 0/1     1            0           3m7s
my-harbor-portal     0/1     1            0           3m7s
my-harbor-registry   0/1     1            0           3m7s
$
```

WAIT ~5 minutes…

kubectl get event -w

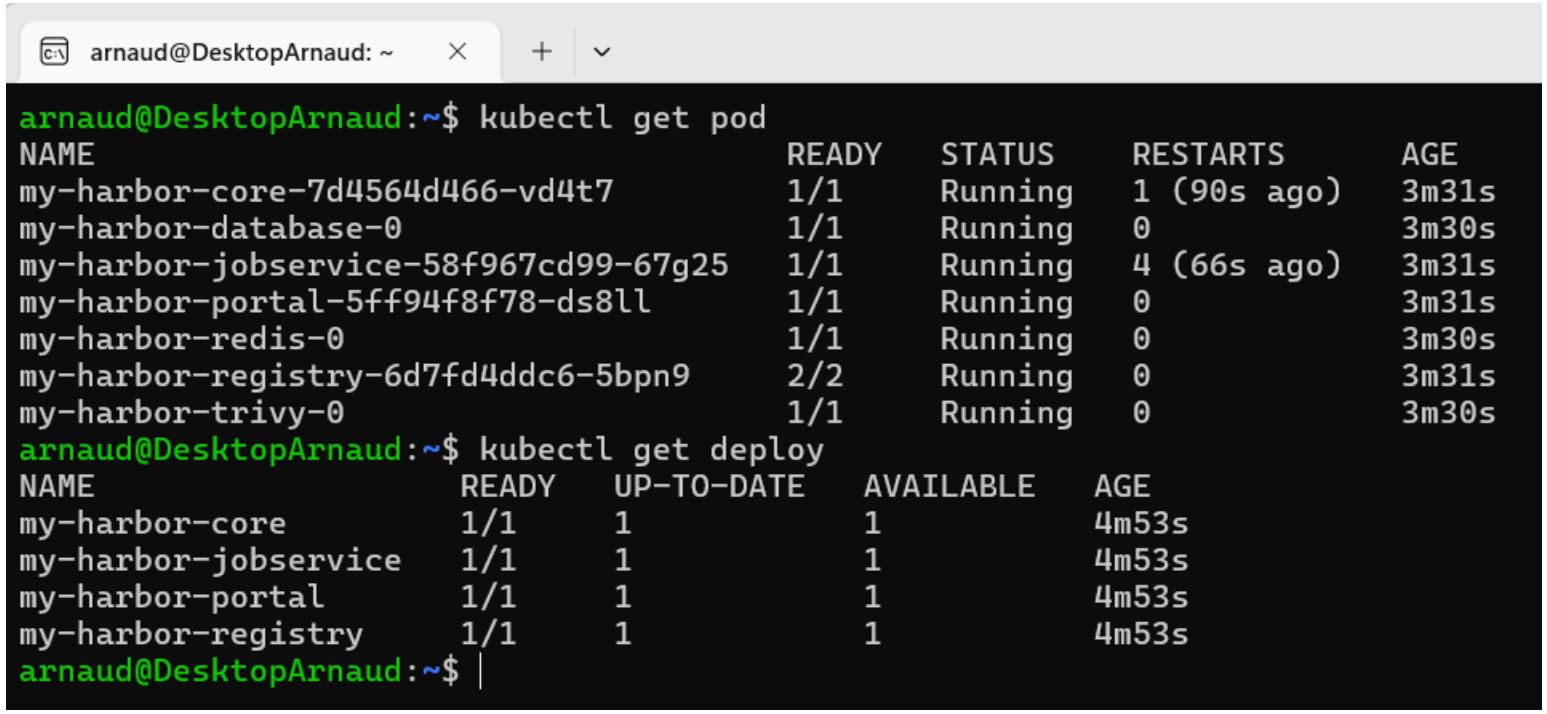
Cmder

```
d harbor/my-harbor-trivy-0 to kind-cluster-control-plane
4m11s       Normal    Pulling                 pod/my-harbor-trivy-0                                  Pulling image "gohar
bor/trivy-adapter-photon:v2.14.2"
12s         Warning   Failed                  pod/my-harbor-trivy-0                                  Failed to pull image
 "goharbor/trivy-adapter-photon:v2.14.2": rpc error: code = DeadlineExceeded desc = failed to pull and unpack image "docker.i
o/goharbor/trivy-adapter-photon:v2.14.2": failed to resolve reference "docker.io/goharbor/trivy-adapter-photon:v2.14.2": fail
ed to do request: Head "https://registry-1.docker.io/v2/goharbor/trivy-adapter-photon/manifests/v2.14.2": dial tcp 52.71.174.
30:443: i/o timeout
12s         Warning   Failed                  pod/my-harbor-trivy-0                                  Error: ErrImagePull
11s         Normal    BackOff                 pod/my-harbor-trivy-0                                  Back-off pulling ima
ge "goharbor/trivy-adapter-photon:v2.14.2"
11s         Warning   Failed                  pod/my-harbor-trivy-0                                  Error: ImagePullBack
Off
8m48s       Normal    SuccessfulCreate        statefulset/my-harbor-trivy                            create Claim data-my
-harbor-trivy-0 Pod my-harbor-trivy-0 in StatefulSet my-harbor-trivy success
8m47s       Normal    SuccessfulCreate        statefulset/my-harbor-trivy                            create Pod my-harbor
-trivy-0 in StatefulSet my-harbor-trivy successful
4m15s       Normal    SuccessfulCreate        statefulset/my-harbor-trivy                            create Pod my-harbor
-trivy-0 in StatefulSet my-harbor-trivy successful
0s          Normal    Pulling                 pod/my-harbor-trivy-0                                  Pulling image "gohar
bor/trivy-adapter-photon:v2.14.2"
```

bash.exe

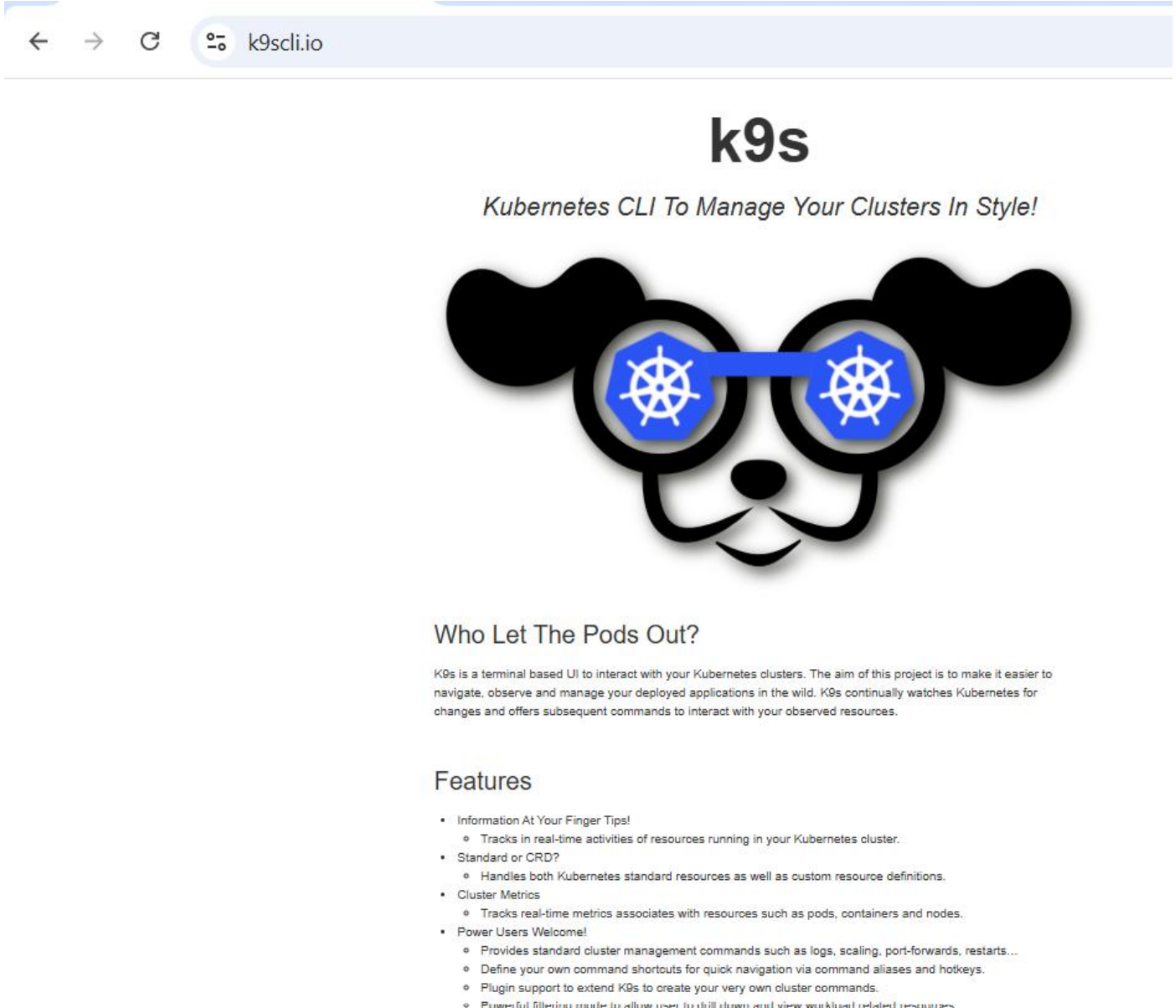
Search

All pods RUNNING , Deployment 1/1 READY



Still Waiting … Install K9s (command line)

https://k9scli.io/



Install K9S on Windows

k9scli.io/topics/install/

k9s

Kubernetes CLI To Manage Your Clusters In Style!

Installation

Overview

K9s is available on Linux, macOS and Windows platforms.

- Binaries for Linux, Windows and Mac are available as tarballs in the release page.
- MacOS

```
# Via Homebrew
brew install derailed/k9s/k9s
# Via MacPort
sudo port install k9s
```

github.com/derailed/k9s/releases

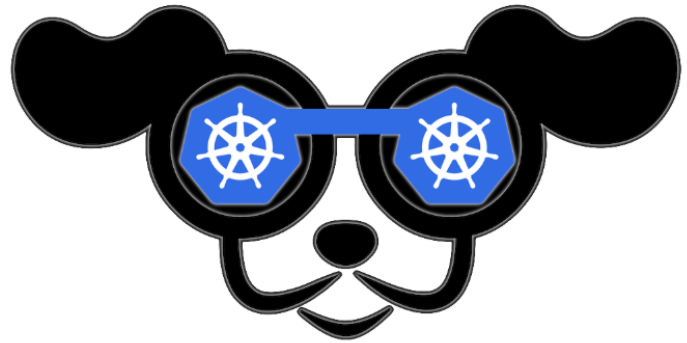
derailed / k9s

Code | Issues 189 | Pull requests 44 | Actions | Projects | Security | Insights

Releases | Tags

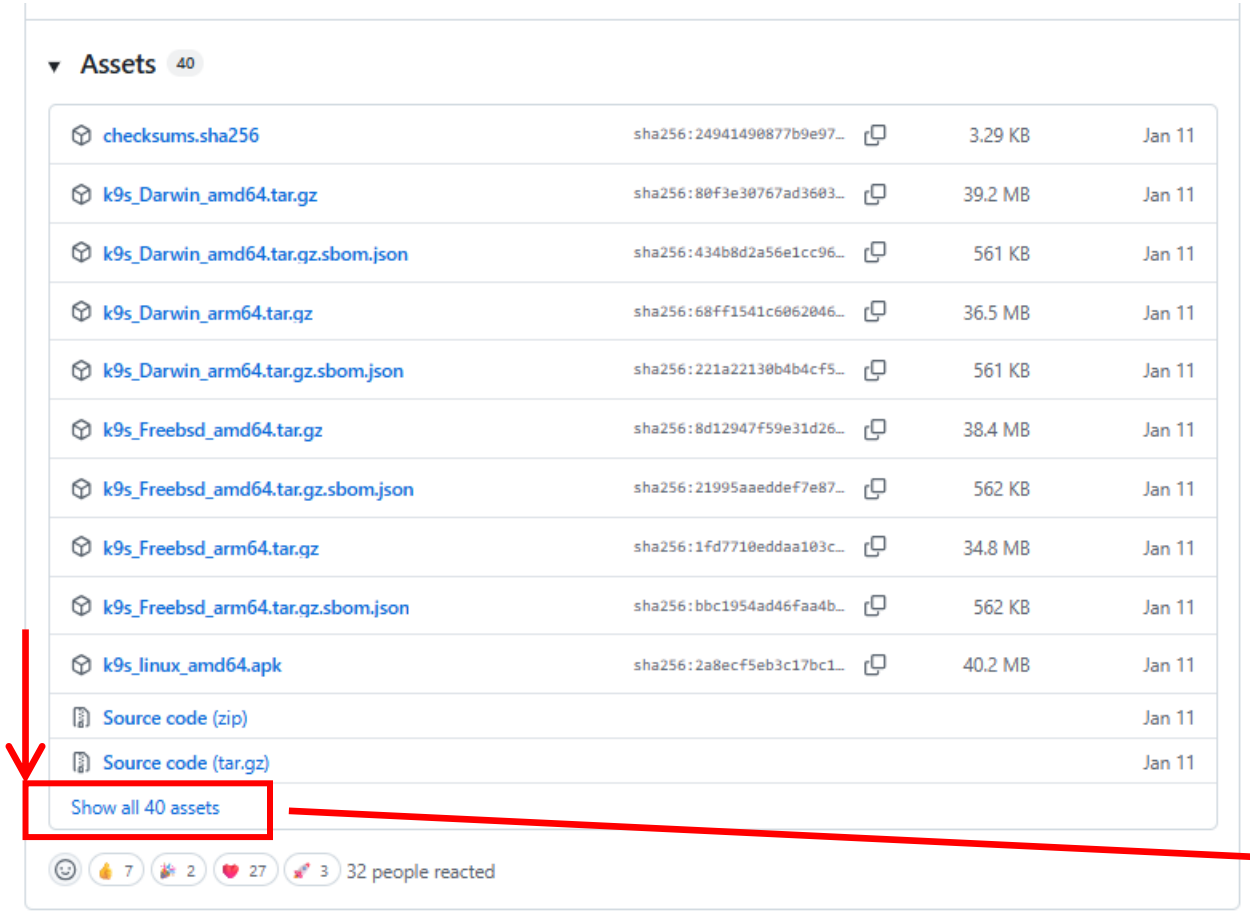
Jan 11
derailed
v0.50.18
6dbf571
Compare

v0.50.18 Latest

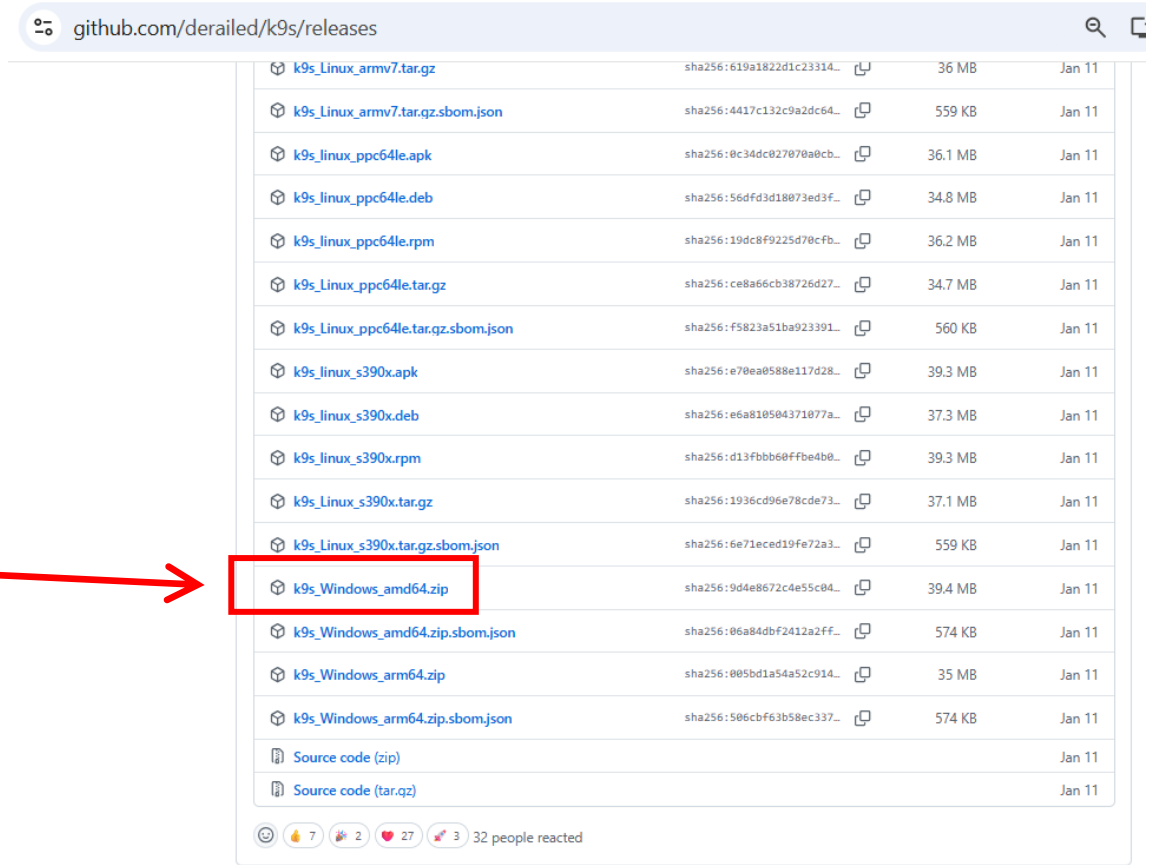


Release v0.50.18

Install K9S on Windows



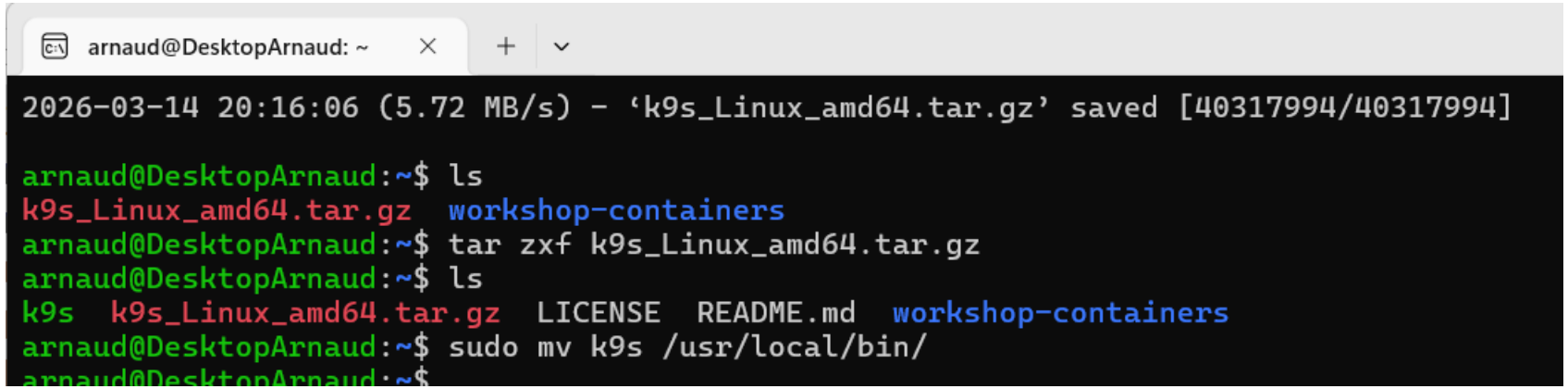




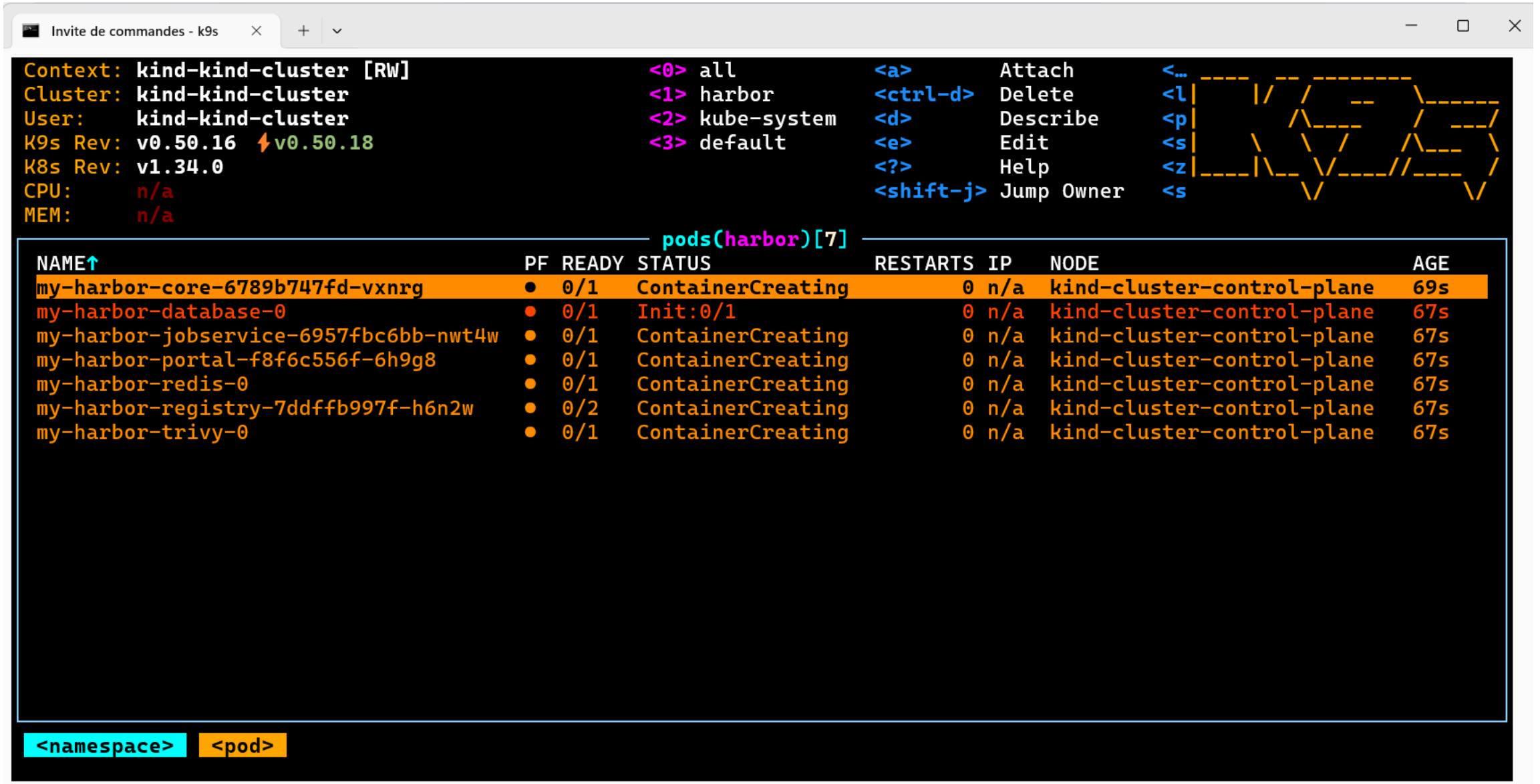


(On Linux) … Install k9s

wget https://github.com/derailed/k9s/releases/download/v0.50.18/k9s_Linux_amd64.tar.gz
tar zxf k9s_Linux_amd64.tar.gz
sudo mv k9s /usr/local/bin/



In terminal, type « K9s »



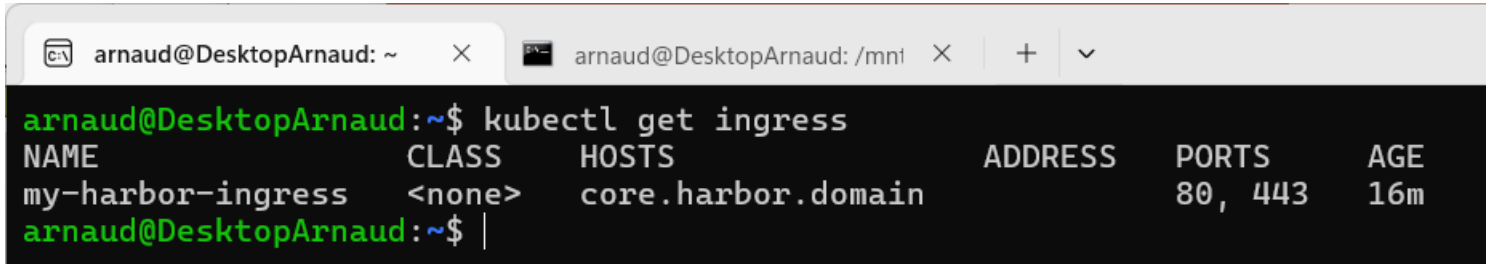
Helm Installed + Installed Tools « k9s »

…

Now Struggle with K8s internals
understand / connect to « http://host:port »

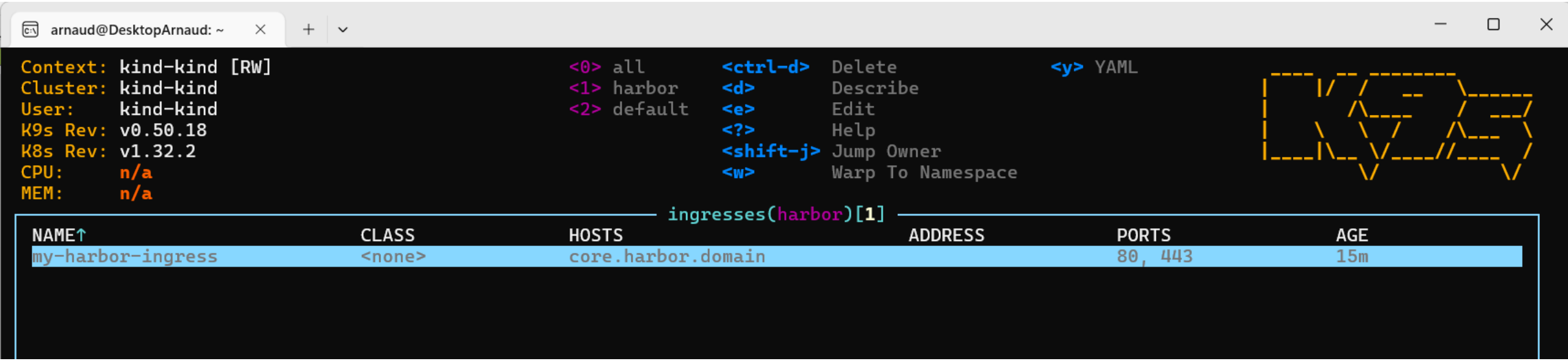
kubectl get ingress

kubectl get ingress



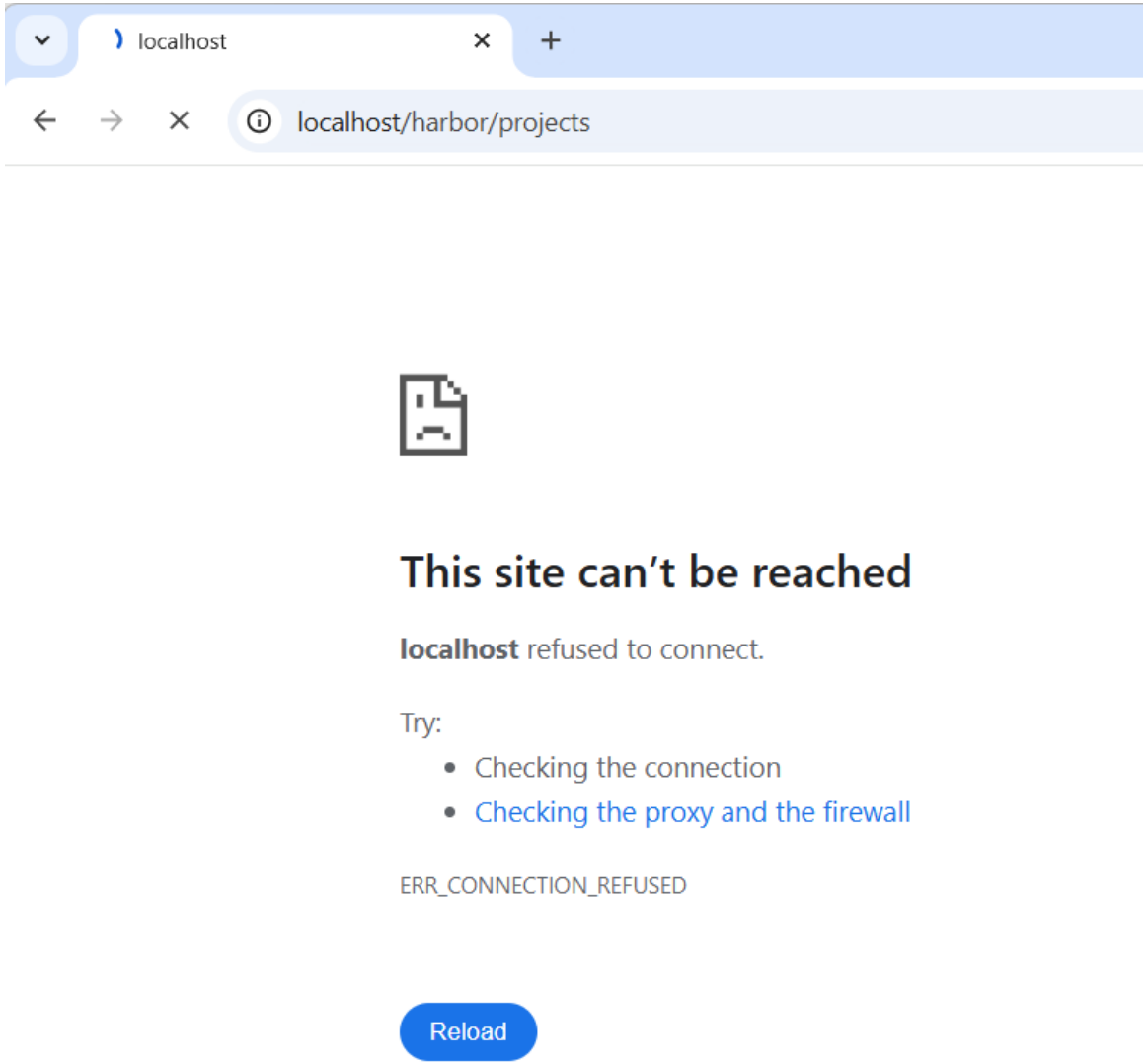
k9s

<ESCAPE> :ingress <ENTER>



http://localhost:80/harbor

=> Connection Refused (= bad port number)



From Podman … Port 8080 on Localhost ?? Give 404 Page Not Found ?!



Podman machine … doing mapping of port !

localhost:9090 -> podman machine 80 (http)
localhost:9443 -> podman machine 443 (https)

podman ps

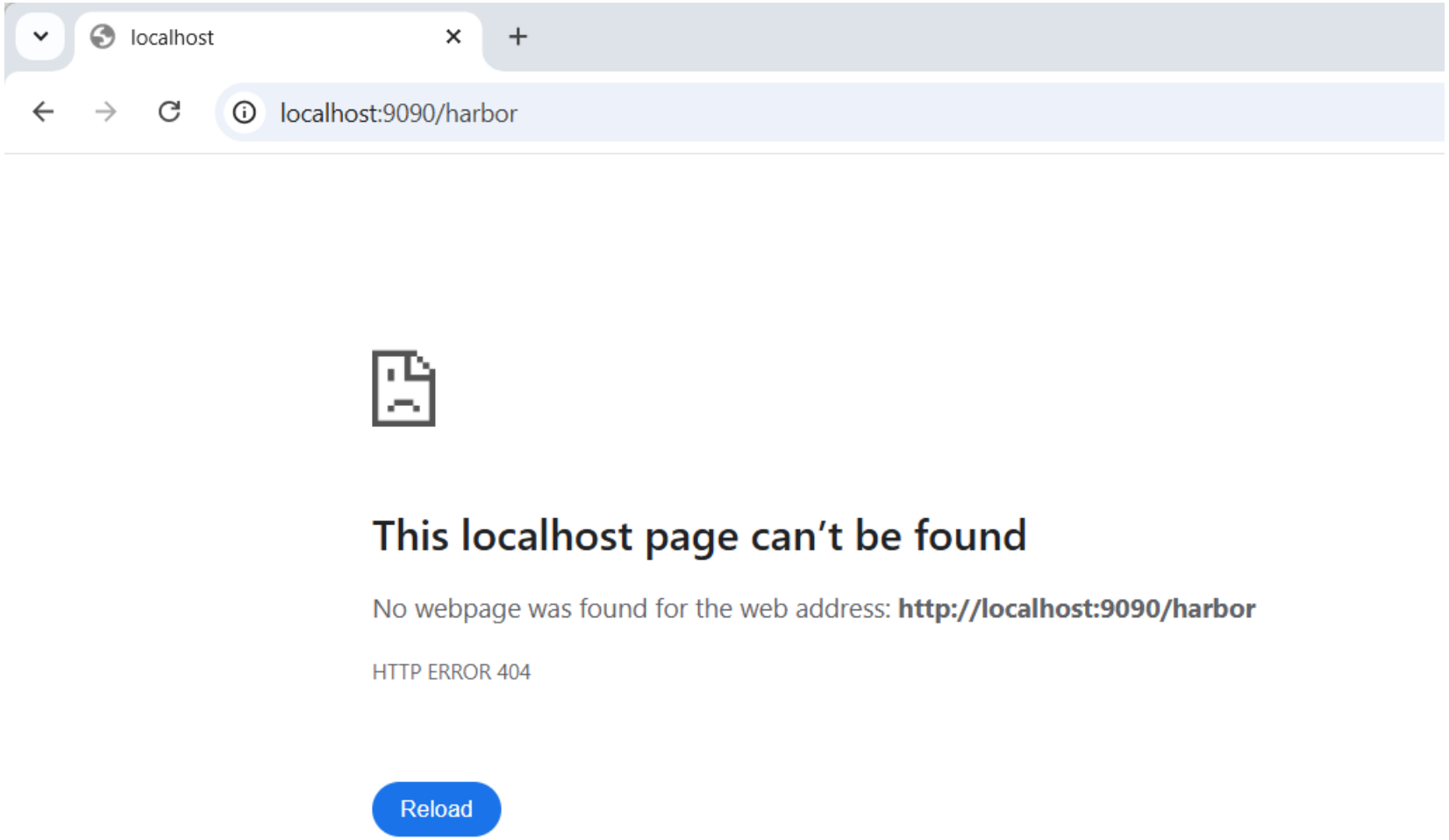
```
C:\Users\arnaud>podman ps
CONTAINER ID  IMAGE                                                                                                                                                COMMAND
  CREATED       STATUS                  PORTS                                                                                                                    NAMES
ee04f2878a22  docker.io/kindest/node@sha256:452d707d4862f52530247495d180205e029056831160e22870e37e3f6c1ac31f
  27 hours ago  Up 7 hours (starting)  0.0.0.0:9090->80/tcp, 0.0.0.0:9443->443/tcp, 127.0.0.1:53334->6443/tcp  kind-clus
ter-control-plane
```

Curl http://localhost:9090/
=> request with http Header
for virtual Host=« localhost:9090»

Cmder

```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -v http://localhost:9090/harbor
* Host localhost:9090 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:9090...
*   Trying 127.0.0.1:9090...
* Established connection to localhost (127.0.0.1 port 9090) from 127.0.0.1 port 60434
* using HTTP/1.x
> GET /harbor HTTP/1.1
> Host: localhost:9090
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< vary: Accept-Encoding
< date: Sun, 15 Mar 2026 21:37:51 GMT
< server: envoy
< content-length: 0
<
* Connection #0 to host localhost:9090 left intact
```

http://localhost:9090/ … 404 NOT_FOUND



Using curl http://localhost:9090
+ forcing http Header for virtual Host « core.harbor.domain »

curl -v http://localhost:9090/harbor -H Host:core.harbor.domain

Cmder

```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -v http://localhost:9090/harbor -H Host:core.harbor.domain
* Host localhost:9090 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:9090...
*   Trying 127.0.0.1:9090...
* Established connection to localhost (127.0.0.1 port 9090) from 127.0.0.1 port 55832
* using HTTP/1.x
> GET /harbor HTTP/1.1
> Host:core.harbor.domain
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< server: envoy
< date: Sun, 15 Mar 2026 21:20:11 GMT
< content-type: text/html
```

Using curl https://localhost:9443
+ forcing http Header for virtual Host « core.harbor.domain

curl -vv -k https://localhost:9443/ -H Host:core.harbor.domain

??? Does not work ???

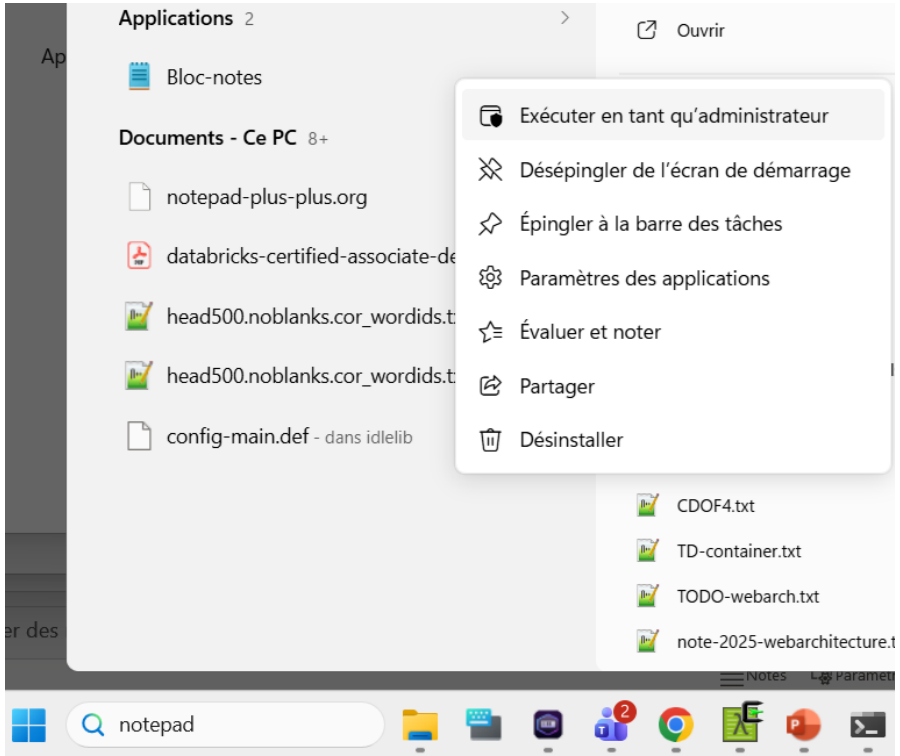
Cmder

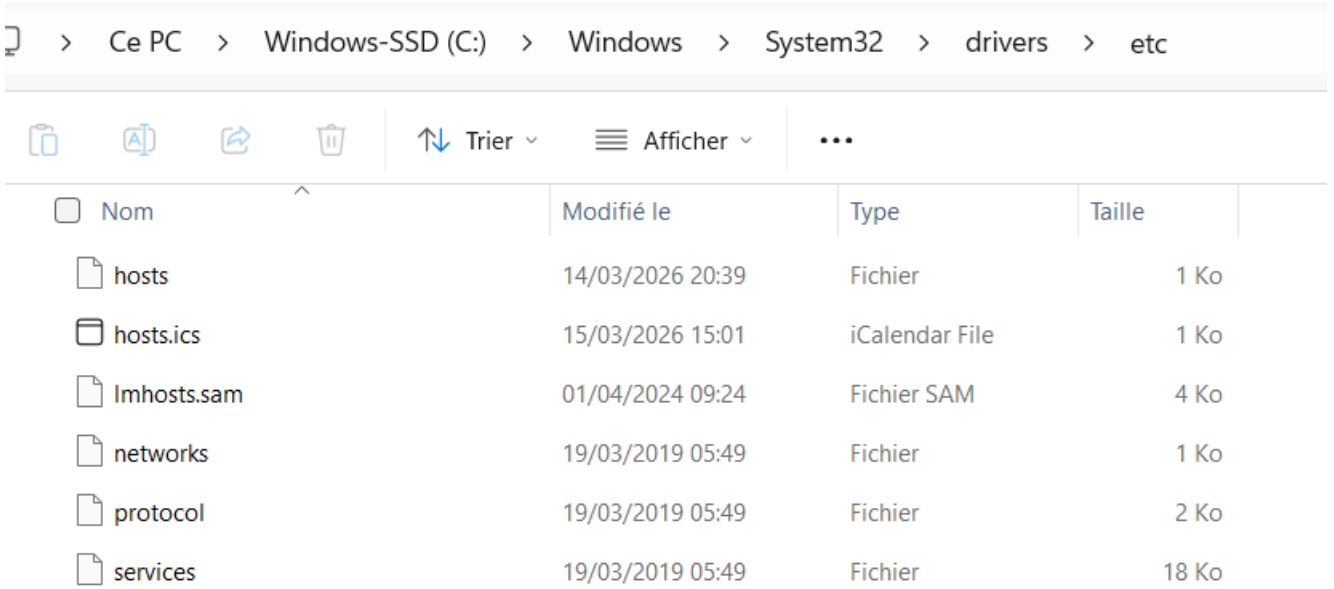
```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -vv -k https://localhost:9443/ -H Host:core.harbor.domain
22:32:50.999000 [0-0] * Host localhost:9443 was resolved.
22:32:51.014000 [0-0] * IPv6: ::1
22:32:51.020000 [0-0] * IPv4: 127.0.0.1
22:32:51.028000 [0-0] * [HTTPS-CONNECT] adding wanted h2
22:32:51.036000 [0-0] * [HTTPS-CONNECT] added
22:32:51.045000 [0-0] * [HTTPS-CONNECT] connect, init
22:32:51.056000 [0-0] *   Trying [::1]:9443...
22:32:51.066000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
22:32:51.074000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
22:32:51.084000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 1 socks
22:32:51.259000 [0-0] *   Trying 127.0.0.1:9443...
22:32:51.270000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
22:32:51.281000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
22:32:51.287000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 2 socks
22:32:51.300000 [0-0] * schannel: disabled automatic use of client certificate
22:32:51.318000 [0-0] * ALPN: curl offers http/1.1
22:32:51.330000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
22:32:51.340000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
22:32:51.352000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 1 socks
22:32:51.362000 [0-0] * schannel: failed to receive handshake, SSL/TLS connection failed
22:32:51.374000 [0-0] * [HTTPS-CONNECT] connect, all attempts failed
22:32:51.389000 [0-0] * [HTTPS-CONNECT] connect -> 35, done=0
22:32:51.404000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 35, done=0
22:32:51.417000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(), filter returned 35
22:32:51.428000 [0-0] * closing connection #0
curl: (35) schannel: failed to receive handshake, SSL/TLS connection failed
```

DNS Hostname « core.harbor.domain » … expecting localhost IP

Solution 1/ …. Force Windows to resolve « core.habor.domain » to 127.0.0.1 (=localhost)

launch Notepad as «Admin» !!
Edit file « **C:\Windows\System32\drivers\etc\hosts**»

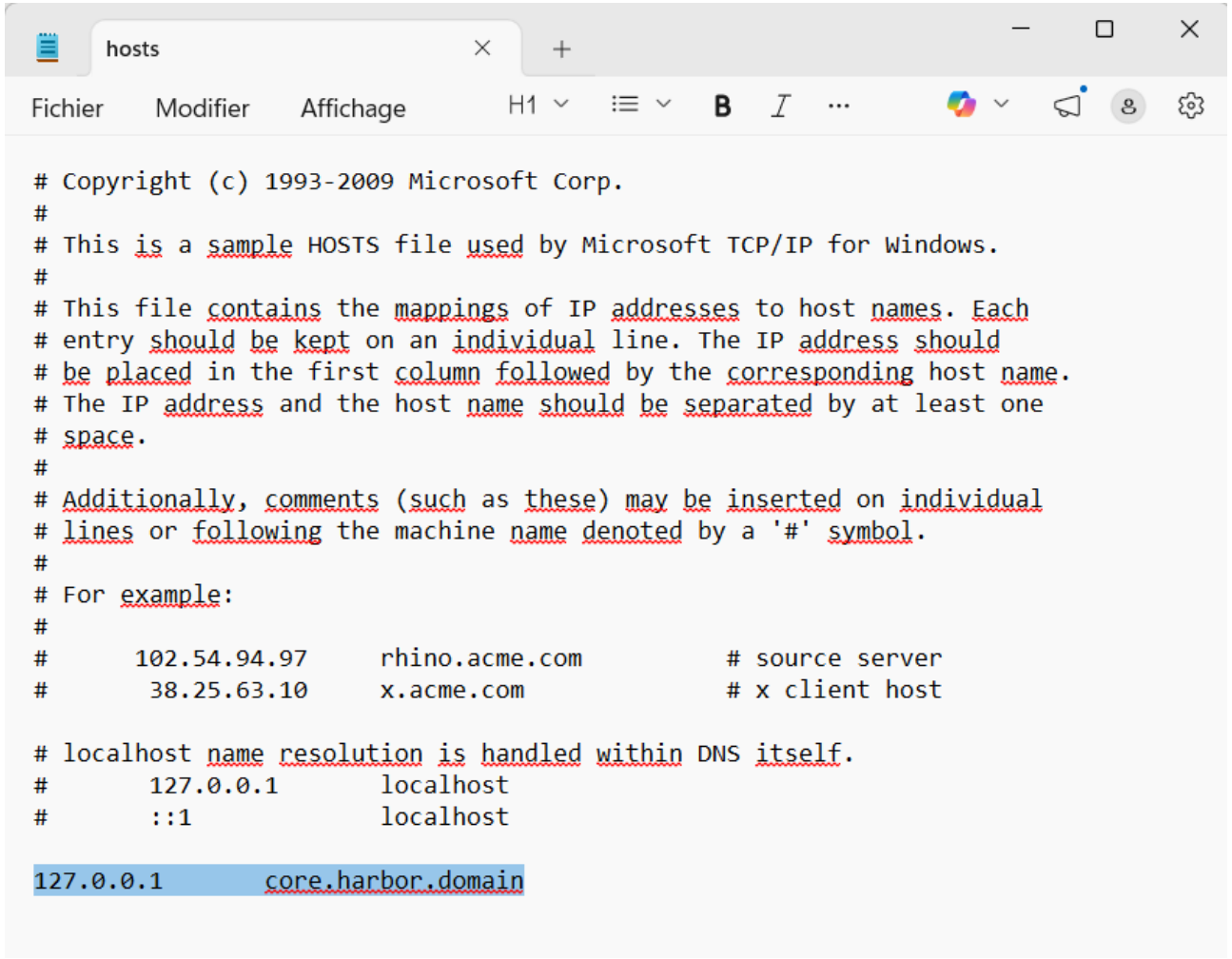




Edit as Admin C:\Windows\System32\drivers\etc\hosts

Add line

127.0.0.1 core.harbor.domain



Check nslookup?? curl

nslookup does not work?? But curl: OK

Cmder

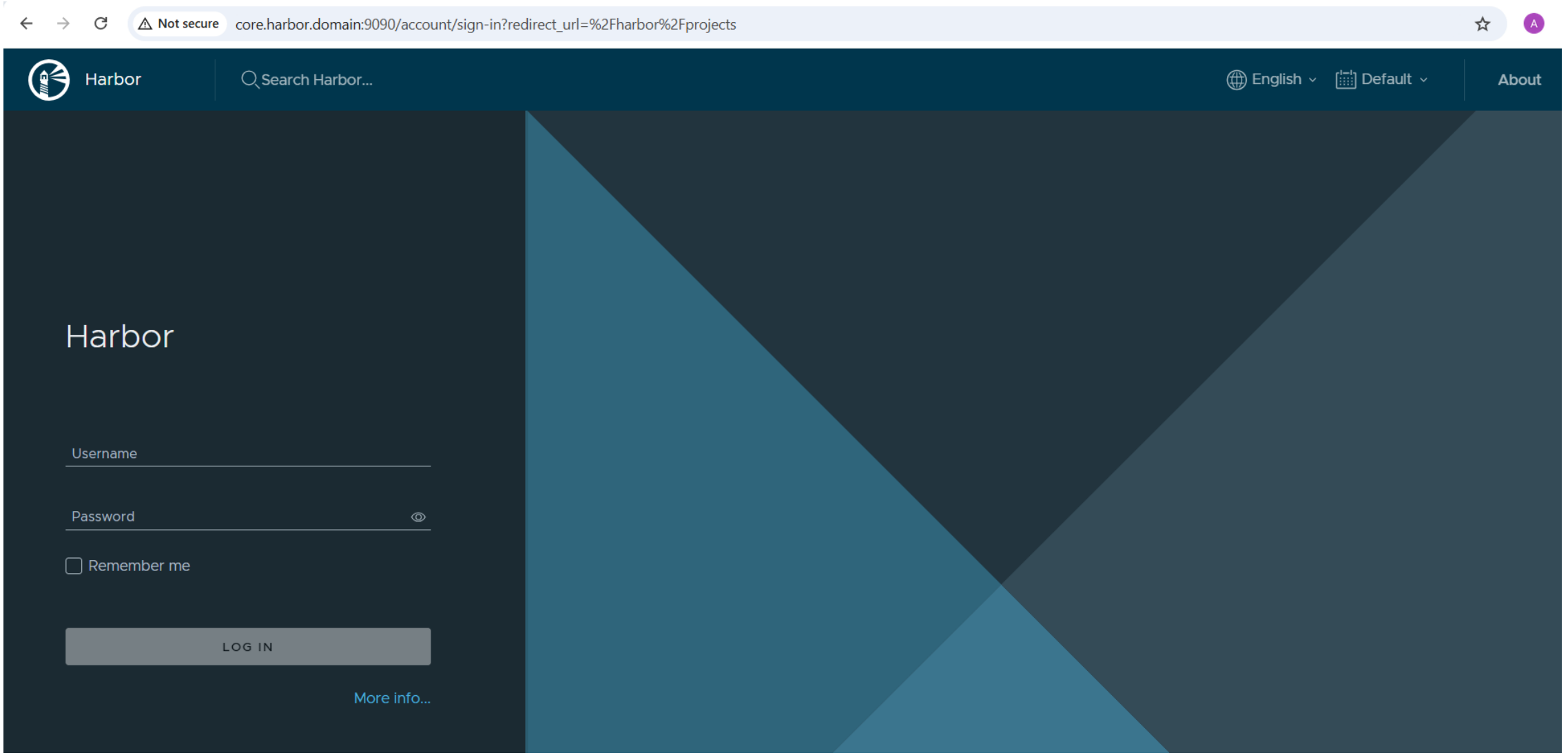
```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ nslookup core.harbor.domain
Serveur :   UnKnown
Address:  192.168.0.254

*** UnKnown ne parvient pas à trouver core.harbor.domain : Non-existent domain

arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -v http://core.harbor.domain:8080/
* Host core.harbor.domain:8080 was resolved.
* IPv6: (none)
* IPv4: 127.0.0.1
*   Trying 127.0.0.1:8080...
* Established connection to core.harbor.domain (127.0.0.1 port 8080) from 127.0.0.1 port 60451
* using HTTP/1.x
> GET / HTTP/1.1
> Host: core.harbor.domain:8080
> User-Agent: curl/8.18.0
> Accept: */*
>
```

http://core.harbor.domain:9090/

It works !!! … redirected to login page

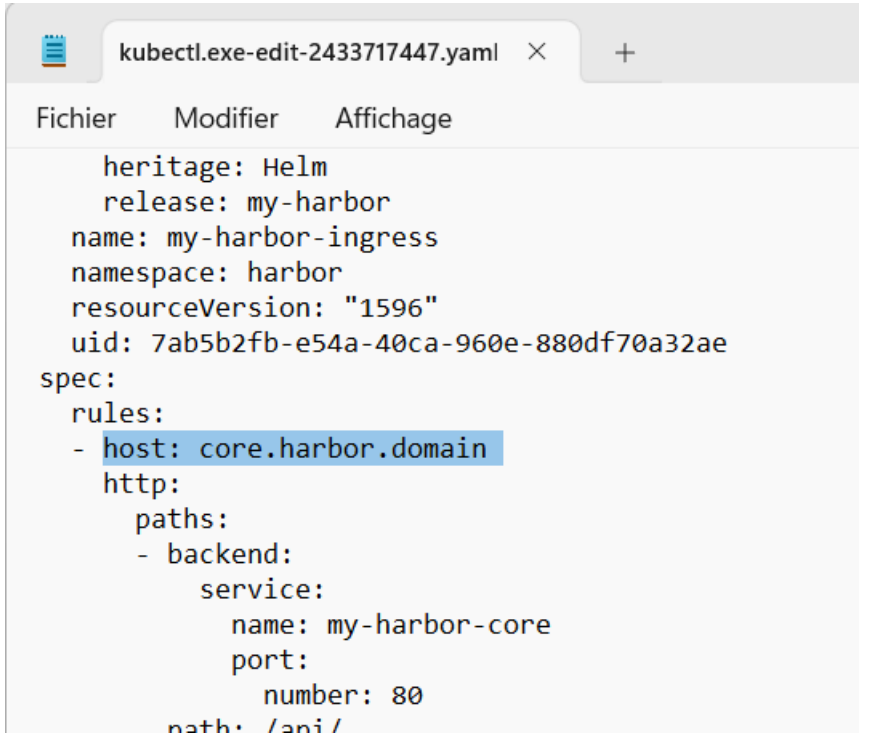


Other possibility… change ingress yaml to expose on hostname: localhost

K9s
<ESCAPE> :ing <e> (=edit)

In editor (notepad), change
host: localhost
Then save / close

This is HACK... Should change param value in Helm
+ do «helm upgrade»



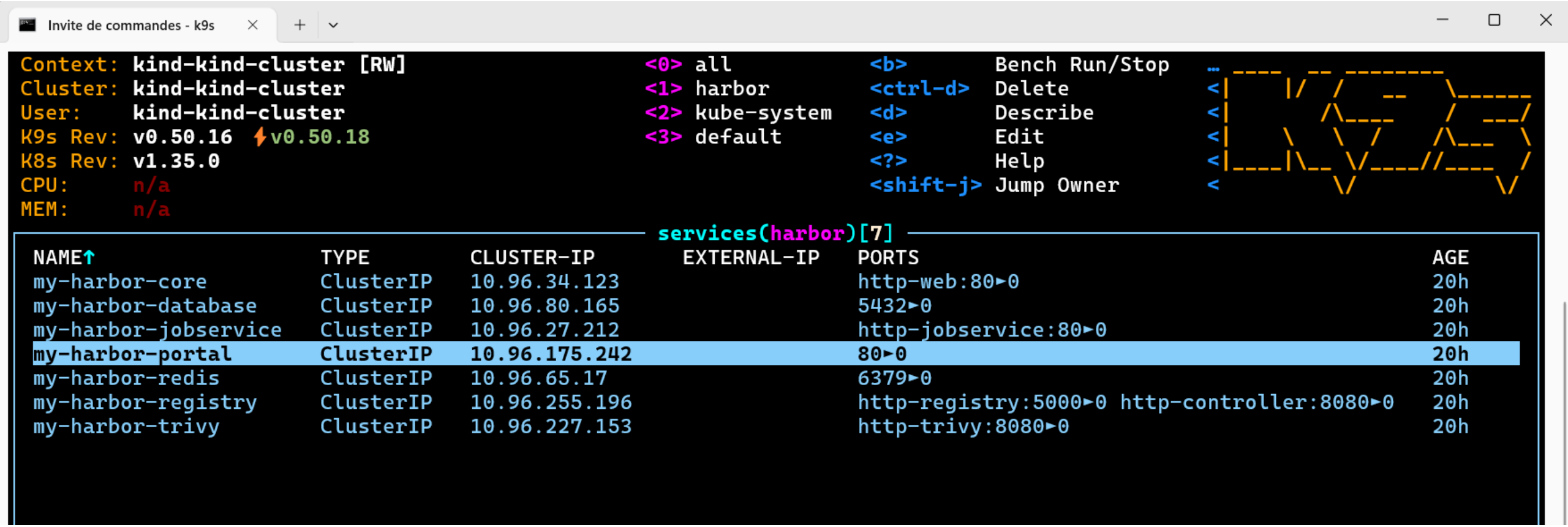
Other Work-Around Alternative for Host:Port

… using Port-Forward !

Find service « portal » … exposing port 8080

k9s
:svc <ENTER> /portal <ENTER>

Equivalent: kubectl get svc



Service details

:svc /portal <ENTER>

<y> (=yaml)

<c> (=copy)

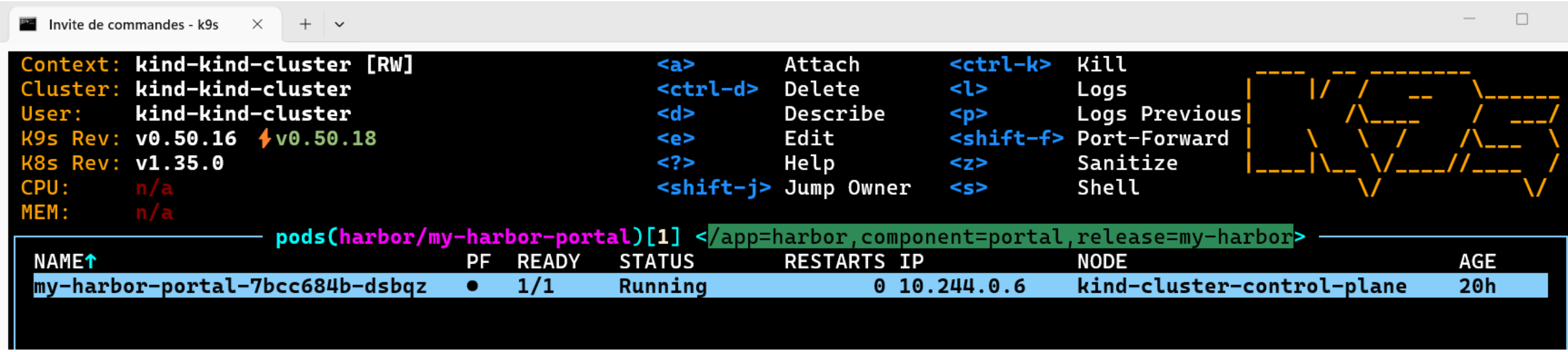
Equivalent: kubectl get svc my-harboor-portal –o yaml

```
Context: kind-kind-cluster [RW]          <c>       Copy                  <f> Toggle Fu…
Cluster: kind-kind-cluster               <e>       Edit                  <m> Toggle Man
User:    kind-kind-cluster               <n>       Next Match
K9s Rev: v0.50.16 ↯v0.50.18              <shift-n> Prev Match
K8s Rev: v1.35.0                         <r>       Toggle Auto-Refresh
CPU:     n/a                             <x>       Toggle Decode
MEM:     n/a
                    YAML(harbor/my-harbor-portal)
  uid: 98b3597f-5c10-4987-b087-697067b1f529
spec:
  clusterIP: 10.96.175.242
  clusterIPs:
  - 10.96.175.242
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
    protocol: TCP
    targetPort: 8080
  selector:
    app: harbor
    component: portal
    release: my-harbor
  sessionAffinity: None
  type: ClusterIP
status:

<service> <yaml>
```

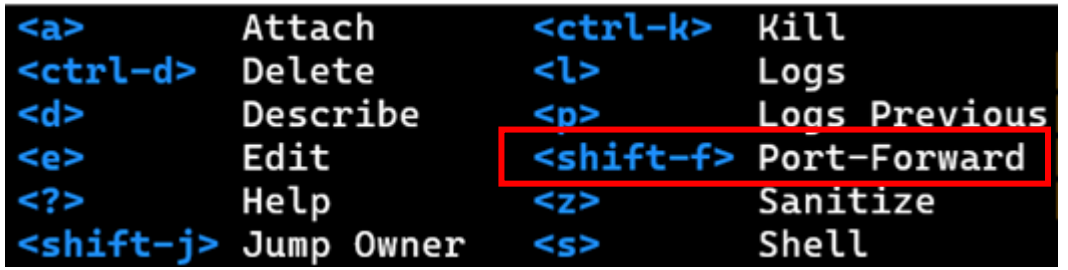
Find pod exposing the « portal » web app

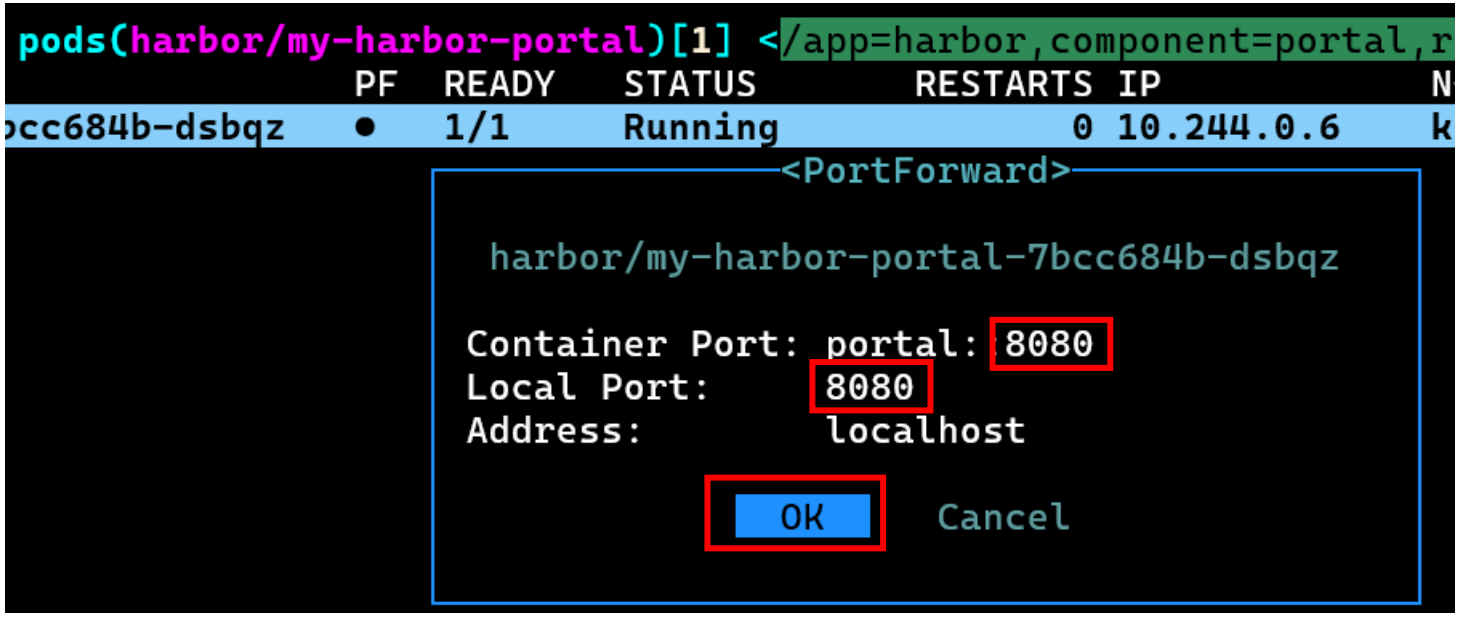
Either from service, type <ENTER>
Or
:pod <ENTER> /portal <ENTER>



Port-Forward 8080:8080

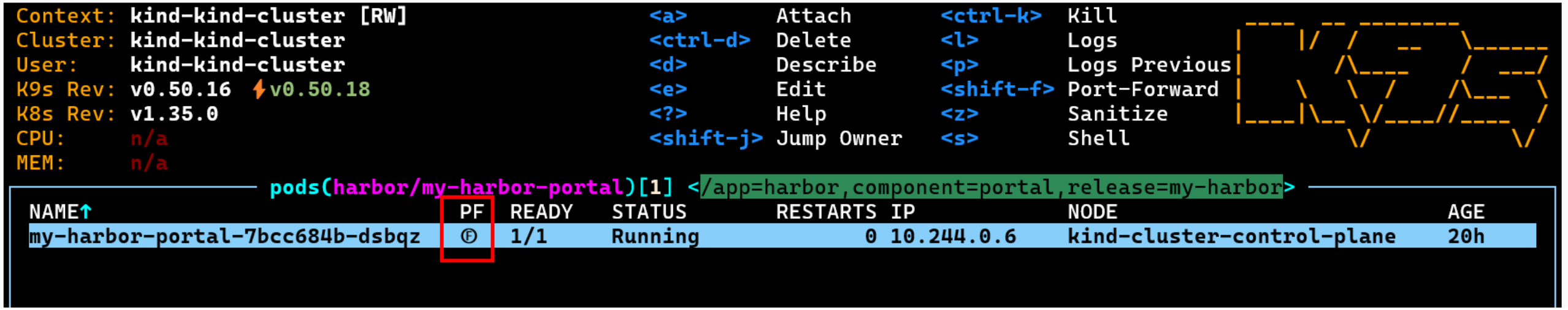
<shift-F> (=port-forward)
<tab> (=8080)
<tab> (=8080)
<ENTER> (=OK)



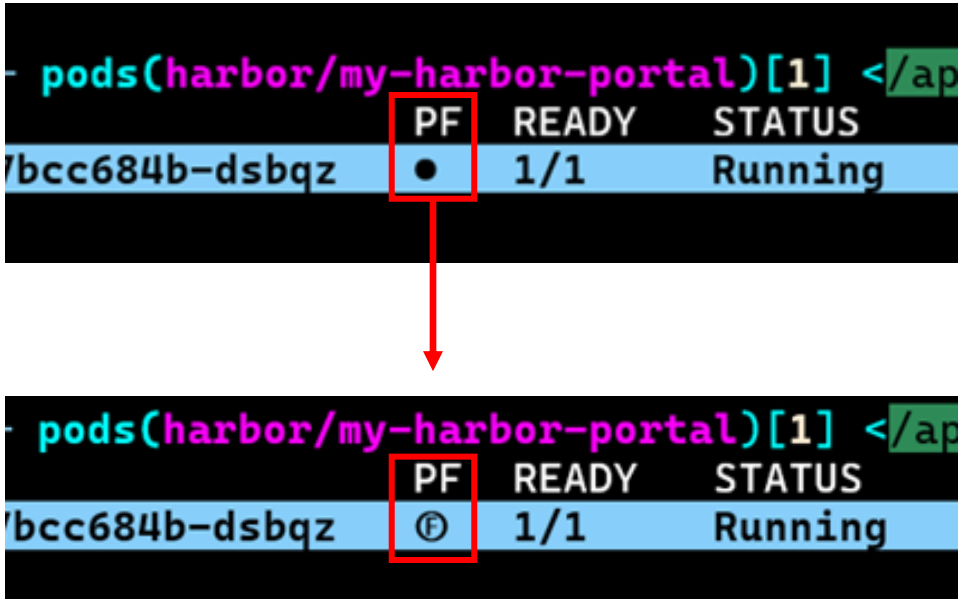


Port Forward Activated

Once done, you see « PF » = « Port-Forward »)



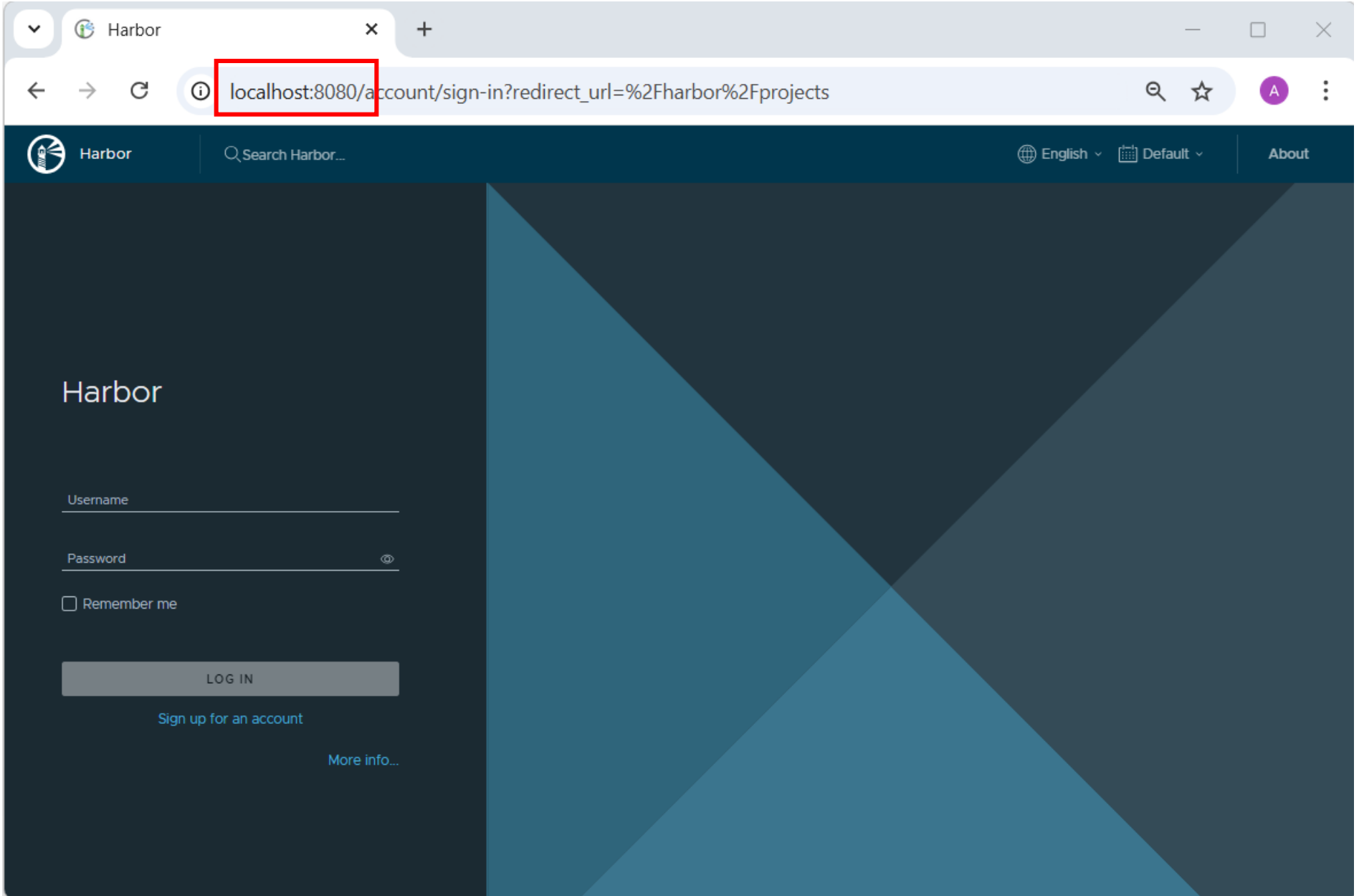
Port-Forward: Activated <-> Stopped if stopping k9s !



Equivalent command: kubectl port-forward

kubectl port-forward my-harbor-portal 8080:8080

http://localhost:8080/harbor

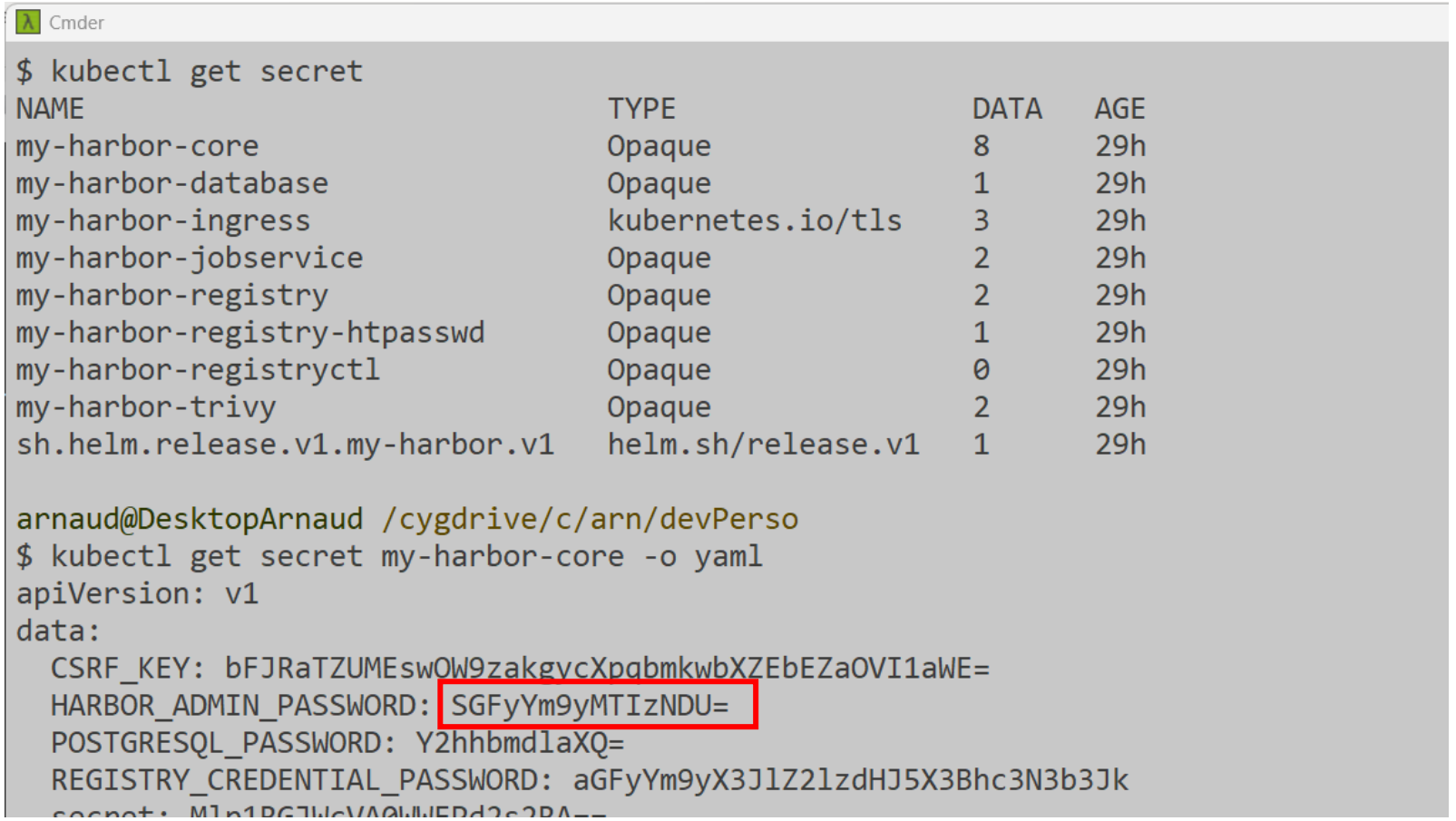


Guessing Login / Password… in k8s secret

Reminder: « admin » / « Harbor12345 » as seen in install.sh via harbor.yml config file

kubectl get secret

kubectl get secret my-harbor-core –o yaml



Secret … encoded in base64 !

echo -n SGFyYm9yMTIzNDU= | base64 -d

```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ echo -n SGFyYm9yMTIzNDU= | base64 -d
Harbor12345
```

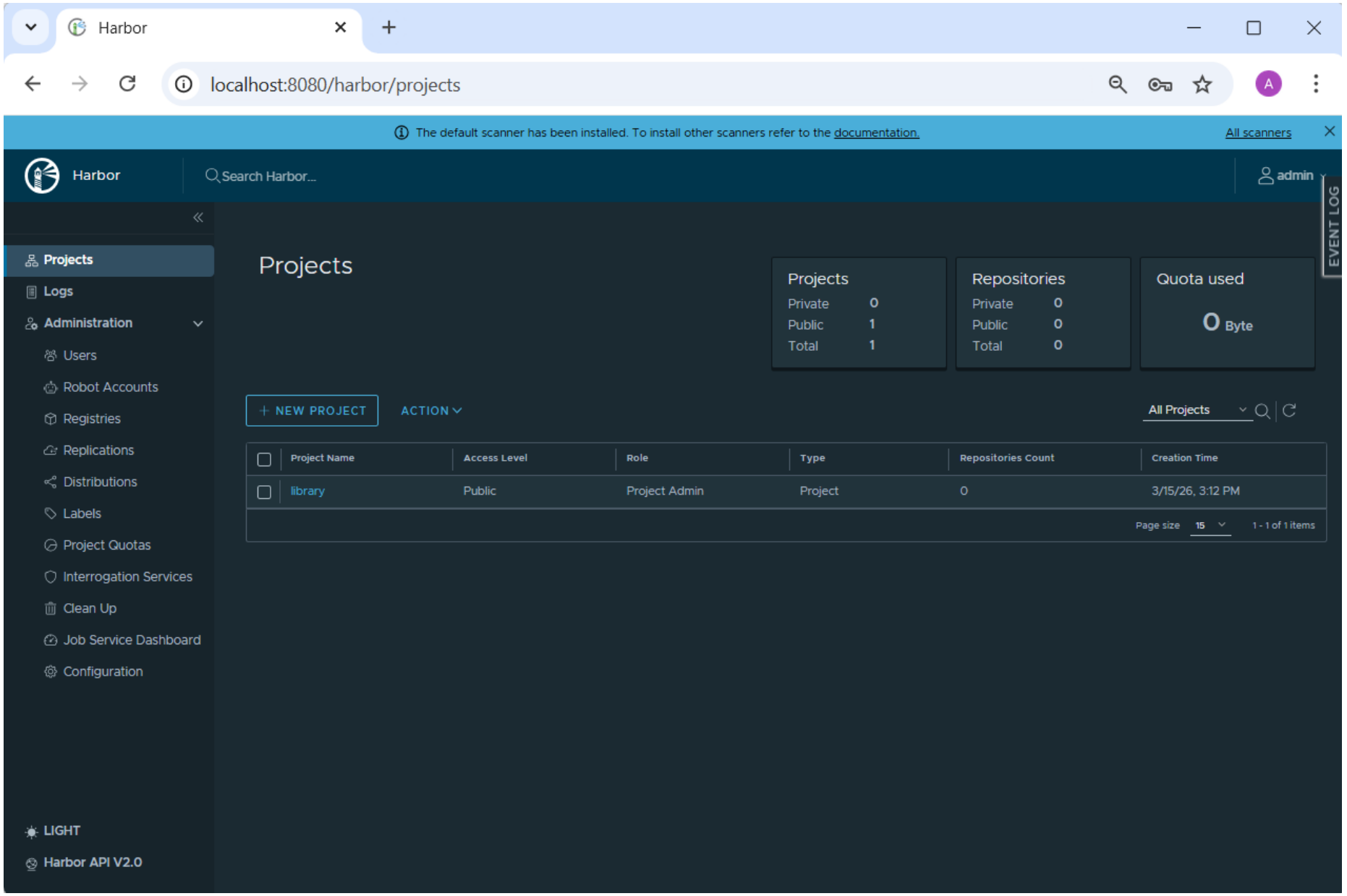
kubectl get secret my-harbor-core -o json | jq -r '.data.HARBOR_ADMIN_PASSWORD'

```
$ kubectl get secret my-harbor-core -o json | jq -r '.data.HARBOR_ADMIN_PASSWORD'
SGFyYm9yMTIzNDU=
```

kubectl get secret my-harbor-core -o json | jq -r '.data.HARBOR_ADMIN_PASSWORD' | base64 -i -d

```
$ kubectl get secret my-harbor-core -o json | jq -r '.data.HARBOR_ADMIN_PASSWORD' | base64 -i -d
Harbor12345
```

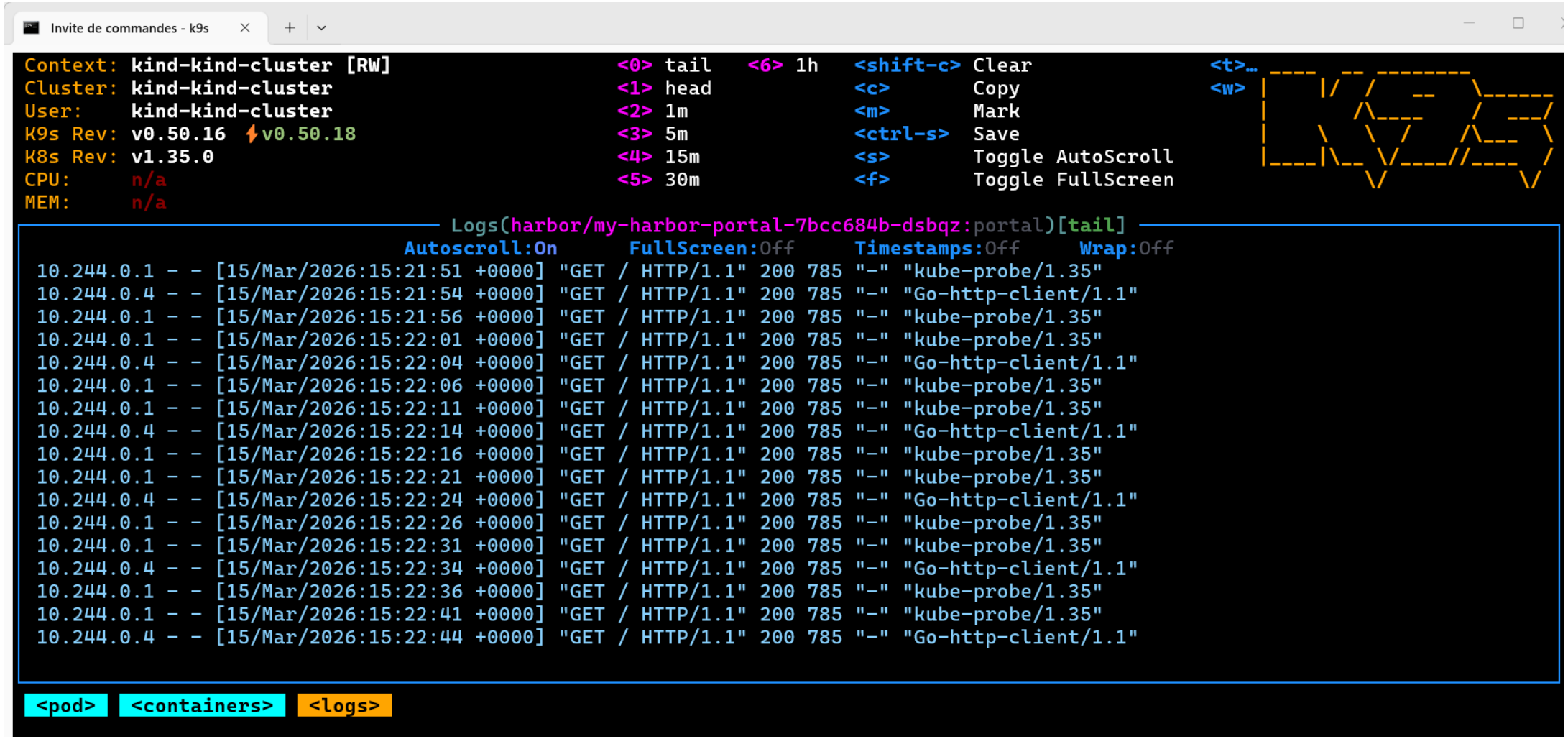
It Works!


Harbor
localhost:8080/harbor/projects
The default scanner has been installed. To install other scanners refer to the documentation.
All scanners
Harbor
Search Harbor...
admin
EVENT LOG
Projects
Logs
Administration
Users
Robot Accounts
Registries
Replications
Distributions
Labels
Project Quotas
Interrogation Services
Clean Up
Job Service Dashboard
Configuration
LIGHT
Harbor API V2.0
Projects
Projects
Private 0
Public 1
Total 1
Repositories
Private 0
Public 0
Total 0
Quota used
0 Byte
+ NEW PROJECT
ACTION
All Projects
Project Name
Access Level
Role
Type
Repositories Count
Creation Time
library
Public
Project Admin
Project
0
3/15/26, 3:12 PM
Page size 15
1 - 1 of 1 items

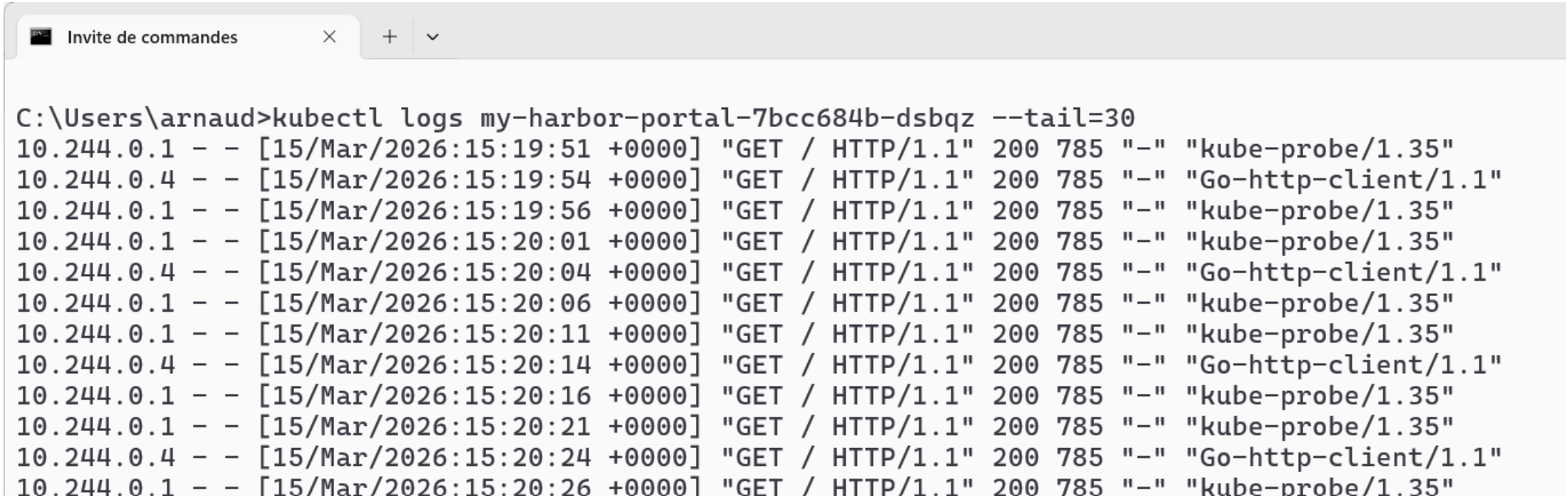
K9s pod logs

K9s :pod /portal

<ENTER> (container => child logs) <ESCAPE> (logs => parent container)

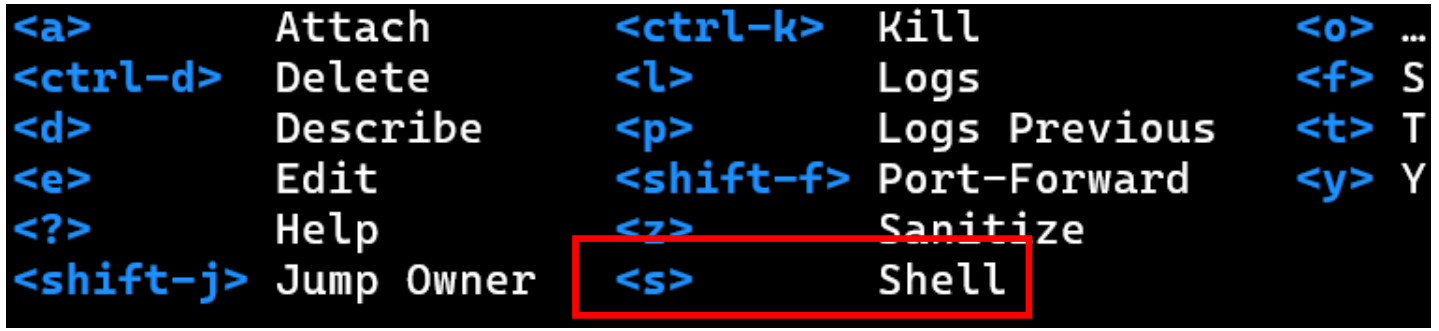


kubectl logs ${pod} [–c ${container}] [-f]

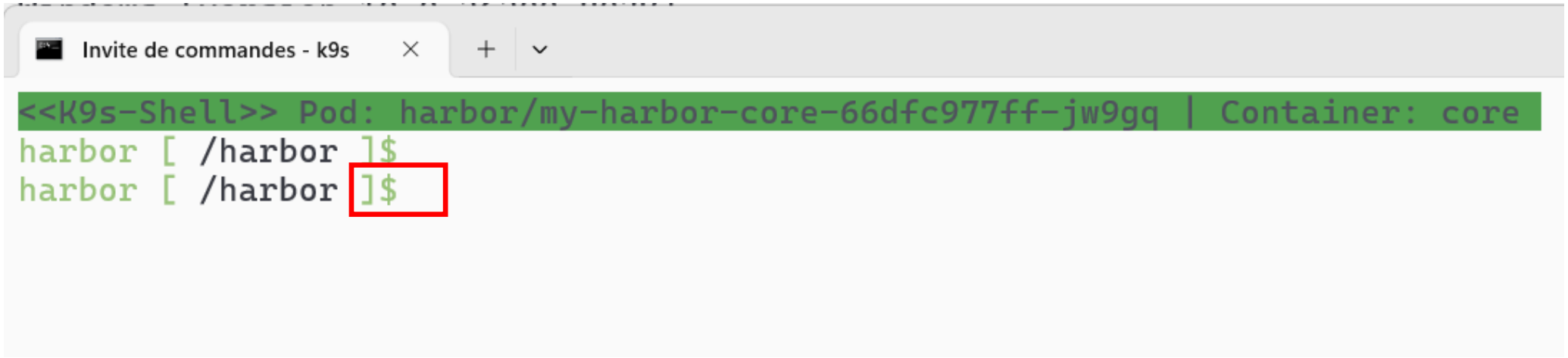


Executing Shell inside Pod

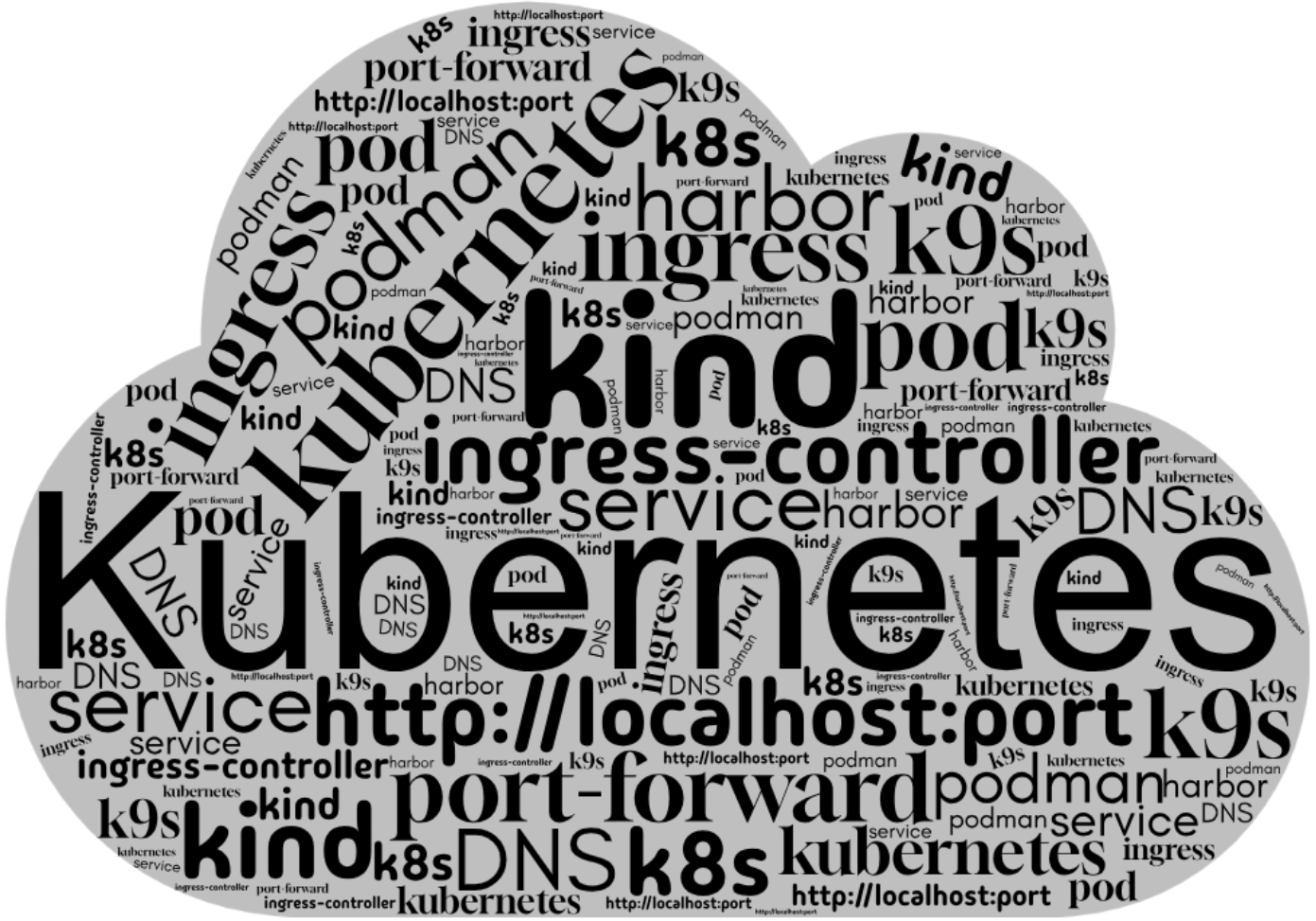
K9s :pod /portal
<s> (=shell)



Equivalent kubectl:
kubectl exec -it my-harbor-portal /bin/sh



Questions?


Kubernetes
kind
ingress
podman
port-forward
http://localhost:port
ingress-controller
service
pod
k9s
k8s
DNS
harbor
kubernetes